

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

COLLEGE NAME: MOTHER TERSA COLLEGE OF ENGINEERING AND
TECHNOLOGY

COLLEGE CODE: 9116

TEAM ID : SWTID-2026-6804

TEAM LEADER : B.MAHESWARAN

EMAIL ID : maheswaranb288@gmail.com

TEAM MEMBER: R Mohamed Ashik

TEAM MEMBER: B Elamurugan

TEAM MEMBER: K Jothishwaran

TEAM MEMBE : A Basith Ahamed

TABLE OF CONTENTS

Section	Page
1. Introduction	7
2. Problem Statement and Existing System	8
3. Proposed System	9
4. Objectives and Scope	10
5. Requirement Analysis	11
6. Feasibility Study	12
7. System Architecture	13
8. Data Model and Database Design	14
9. Multi-Line Policy Configuration	15
10. Policy Lifecycle	16
11. Premium Calculation and Auto Quoting	17
12. Claims Management	18
13. Claim Routing and Approval	19
14. Claims Adjuster LWC Dashboard	20
15. Apex and SOQL Implementation	21
16. Salesforce Flow Automation	22
17. Security and Access Control	23
18. User Interface Design	24
19. Testing Strategy	25
20. Test Cases and Results	26
21. Deployment and Maintenance	27
22. Benefits, Limitations and Future Scope	28
23. Conclusion	29
24. References and Appendix	30

1. INTRODUCTION

1.1 Background

Insurance organizations manage customers, policies, premiums, renewals, endorsements, claims, approvals, settlements, and supporting documents. When these activities are handled through disconnected spreadsheets or manual workflows, organizations may experience duplicated information, inconsistent processing, delayed approvals, and limited visibility.

1.2 Need for a Multi-Line Platform

A multi-line insurance environment contains different products with different data requirements. An Auto policy may require vehicle identification and model year, while a Property policy may require square footage and property characteristics. Life insurance may require insured-person information and coverage details. A common platform must support these differences without creating an unmanageable collection of unrelated applications.

1.3 Salesforce Approach

Salesforce provides configurable data objects, record types, automation, security, reporting, and programmable components. These capabilities allow the project to model insurance operations as an integrated business application.

1.4 Project Theme

The project combines declarative configuration and programmatic development. Declarative features handle standard business workflows, while Apex and LWC are used where custom calculations, data retrieval, or user experiences are required.

2. PROBLEM STATEMENT AND EXISTING SYSTEM

2.1 Problem Statement

The project addresses the difficulty of managing multiple insurance products and claims through a unified, traceable workflow. The system must support different policy types, calculate or update premium information, capture claims, route claims to responsible users, and provide adjusters with relevant information.

2.2 Existing-System Challenges

- Customer and policy information may be duplicated across systems.
- Different insurance lines often use separate forms and data structures.
- Manual premium calculations can introduce inconsistent values.
- Claims may be routed through email or informal communication.
- Approval status can be difficult to track.
- Adjusters may need to search multiple screens to understand a claim.
- Management may lack a single operational view of claims and policies.
- Changes may be difficult to audit when processes are not automated.

2.3 Business Impact

These challenges can increase processing time and operational effort. They can also make it harder to identify bottlenecks and maintain consistent business rules. A centralized platform provides a common foundation for standardization and visibility.

3. PROPOSED SYSTEM

3.1 Overview

The proposed system provides a centralized Salesforce application for policy and claims administration. It uses a Policy custom object with Auto, Property, and Life record types and a Claim custom object with claim-specific fields and approval status.

3.2 Major Capabilities

Module	Main Capability
Customer Management	Maintain customer/insured information and relationships.
Policy Management	Create and manage policies across multiple insurance lines.
Auto Policy	Capture vehicle-specific information such as VIN and model year.
Property Policy	Capture property-specific information such as square footage.
Life Policy	Maintain life coverage and insured-person information.
Quoting	Guide users through quote data collection and premium calculation.
Claims	Create, track, route, review, and approve claims.
Adjuster Dashboard	Present logged-in adjuster claims and related information.
Approval	Use sequential senior-adjuster and department-manager review.
Security	Apply permissions, sharing, validation, and role-based access.

3.3 Expected Result

The result is a configurable enterprise application that improves information consistency and gives different users an appropriate view of their work.

4. OBJECTIVES AND SCOPE

4.1 Objectives

1. Create a unified data model for multi-line insurance policies.
2. Differentiate Auto, Property, and Life policies using record types.
3. Use field sets to expose policy-specific information.
4. Create a structured claims management process.
5. Automate premium calculation using Apex.
6. Integrate the premium calculation with an automated quote flow.
7. Route claims through a sequential approval process.
8. Develop a Claims Adjuster LWC dashboard.
9. Retrieve related Claim, Policy, and Customer information using SOQL.
10. Improve traceability through controlled statuses and Salesforce security.

4.2 Scope

The scope covers policy configuration, claim configuration, automation, approval, dashboard development, security design, testing, and deployment considerations. It is suitable as a prototype or academic enterprise application.

4.3 Out of Scope

- Real-world underwriting decisions
- Binding legal insurance contracts
- Production actuarial rating certification
- Actual claim payment processing
- Government/regulatory integration
- Production fraud investigation

5. REQUIREMENT ANALYSIS

5.1 Functional Requirements

- The system shall create and maintain policy records.
- The system shall support Auto, Property, and Life policy types.
- The system shall provide policy-specific fields.
- The system shall create and maintain claim records.
- The system shall associate claims with relevant policies and customers.
- The system shall maintain claim approval status.
- The system shall calculate/update premium information through Apex.
- The system shall support an automated quoting flow.
- The system shall route claims for sequential approval.
- The system shall provide adjusters with a dashboard of their claims.

5.2 Non-Functional Requirements

Requirement	Target
Performance	Efficient SOQL and server-side processing.
Security	Role/profile/permission-based access.
Usability	Clear screens and guided workflows.
Maintainability	Reusable Apex and modular LWC.
Scalability	Support additional insurance lines.
Auditability	Track statuses, approvals, and record changes.
Reliability	Validation and controlled automation.

5.3 User Roles

Typical roles include customer-facing users, agents, claims adjusters, senior adjusters, department managers, and system administrators.

6. FEASIBILITY STUDY

6.1 Technical Feasibility

The system is technically feasible because Salesforce supplies the required platform services for structured data, automation, server-side logic, user interfaces, authentication, permissions, and reporting. Apex, SOQL, Flow, and LWC cover the core implementation requirements.

6.2 Operational Feasibility

The proposed workflow maps naturally to insurance operations: create policy, collect information, calculate premium, manage active policy, create claim, route claim, review, approve/reject, and monitor.

6.3 Economic Feasibility

A cloud platform can reduce the need to build and maintain infrastructure from scratch. For an academic prototype, Salesforce also provides a controlled environment for demonstrating enterprise features. Actual production costs depend on licenses, integrations, data volumes, support, and compliance requirements.

6.4 Schedule Feasibility

Milestone	Focus
Milestone 1	Core data model and policy configuration
Milestone 2	Complex policy insurance and claim routing automation
Milestone 3	Claims Adjuster LWC dashboard
Milestone 4	Advanced claim processing, security and code setup

The supplied project screenshots indicate progress across these four milestones.

7. SYSTEM ARCHITECTURE

The architecture follows a layered enterprise pattern. Users interact with Salesforce Lightning pages and the custom LWC dashboard. Declarative automation handles guided workflows, while Apex handles specialized server-side calculations and data retrieval.

Layer	Components	Purpose
Presentation	Lightning App, LWC Dashboard	User interaction and visualization
Automation	Flow, Approval Process	Guided processes and routing
Business Logic	Apex classes	Premium calculation and claim services
Data	Policy, Claim, Customer records	Persistent insurance information
Security	Profiles, Permissions, Sharing	Controlled access
Reporting	Reports/Dashboards	Operational visibility

7.1 High-Level Flow

Step	Process
1	User creates or selects customer.
2	User creates policy using an appropriate record type.
3	Policy-specific fields capture line-specific information.
4	Quote flow gathers inputs and invokes premium logic.
5	Policy record is updated with calculated premium.
6	Claim is created when a loss/event is reported.
7	Claim is routed to responsible adjuster.
8	Sequential approval is performed.
9	Adjuster dashboard displays claim information.
10	Claim status and outcome are recorded.

8. DATA MODEL AND DATABASE DESIGN

8.1 Core Objects

Object	Key Purpose
Policy	Stores insurance policy information.
Claim	Stores reported loss and claim processing information.
Customer	Stores insured/customer information.
Vehicle/Auto Data	Logical group of vehicle-specific policy fields.
Property Data	Logical group of property-specific fields.
Life Data	Logical group of life-insurance-specific fields.

8.2 Policy Record Types

The Policy object uses record types to represent Auto, Property, and Life insurance. Record types allow the same object to support different business processes and page layouts.

8.3 Field Sets

The project uses field sets to organize policy-specific fields. The supplied milestone material specifically identifies Auto_Fields for data such as VIN and Model Year and Property_Fields for data such as Square Footage.

8.4 Claim Relationship

A Claim should reference the relevant Policy and customer/insured information. This relationship enables the Claims Adjuster dashboard to show claim context without requiring users to manually search for related records.

9. MULTI-LINE POLICY CONFIGURATION

9.1 Auto Insurance

Auto policies require vehicle-oriented information. Typical fields include VIN, model year, make, model, vehicle usage, coverage limits, deductible, and driver information. In the project, the Auto record type and Auto_Fields field set provide the structure for vehicle-specific data.

9.2 Property Insurance

Property policies require information about the insured property. Typical data includes property type, address, construction details, occupancy, square footage, value, coverage limit, deductible, and risk characteristics. The supplied project milestone identifies Square Footage as a Property-specific field.

9.3 Life Insurance

Life policies focus on the insured person and coverage. Typical fields include insured name, coverage amount, policy term, beneficiary information, premium frequency, and policy status.

9.4 Advantages of Record Types

- One Policy object can support multiple products.
- Different layouts and picklists can be configured per product.
- Users see relevant fields instead of unrelated information.
- Future insurance lines can be added without replacing the core Policy object.
- Reporting can combine or segment policies by record type.

10. POLICY LIFECYCLE

10.1 Lifecycle Stages

Stage	Description
Lead/Request	Customer expresses interest in an insurance product.
Quote	Policy information is collected and premium is estimated.
Review	Business rules and required data are checked.
Draft	Policy record is prepared for activation.
Active	Policy becomes an active insurance record in the system.
Renewal	Policy is reviewed for continuation.
Endorsement	Permitted policy details are changed.
Expired/Cancelled	Policy is no longer active.

10.2 Data Validation

Validation rules should prevent incomplete or invalid information. For example, Auto policies can require VIN and Model Year; Property policies can require Square Footage. Required fields and validation rules should be aligned with the record type and business process.

10.3 Auditability

Status fields, created/modified timestamps, approval history, and field history tracking can help organizations understand what happened to a policy and when.

11. PREMIUM CALCULATION AND AUTO QUOTING

11.1 Premium Calculator

The project milestone specifies an Apex class named PremiumCalculator with a method that accepts policy data. The class represents a reusable server-side service for calculating or determining premium values.

11.2 Conceptual Calculation

A simplified educational model can be expressed as: $\text{Base Premium} + \text{Risk Adjustment} + \text{Coverage Adjustment} - \text{Applicable Discount} = \text{Estimated Premium}$. A real insurer would use actuarial models, regulatory rules, underwriting factors, rating tables, and product-specific calculations.

11.3 AutoQuotingFlow Integration

The Apex action is integrated into the final step of the AutoQuotingFlow so that the calculated value can be written to the draft Policy record. This separates user-guided data collection from reusable calculation logic.

11.4 Example Flow

11. Start AutoQuotingFlow.
12. Collect customer and vehicle information.
13. Validate required fields.
14. Create or identify the draft Policy.
15. Invoke PremiumCalculator Apex action.
16. Receive the calculated premium.
17. Update the draft Policy record.
18. Display or save the quote result.
19. End the flow.

This approach provides a clear boundary between declarative process orchestration and custom business logic.

12. CLAIMS MANAGEMENT

12.1 Claim Creation

A claim represents a reported insured event. The Claim object should capture claim number, policy reference, claimant/customer, date of loss, description, estimated amount, claim type, status, assigned adjuster, and approval status.

12.2 Claim Lifecycle

Stage	Typical Activity
Reported	Claim is registered.
Assigned	Claim is assigned to an adjuster.
Under Review	Evidence and policy coverage are evaluated.
Assessment	Adjuster evaluates loss information.
Approval	Required management review is completed.
Decision	Claim is approved, rejected, or requires further action.
Closed	Claim is completed and retained for reference.

12.3 Claim Data Quality

- Require a valid policy relationship.
- Capture date and description of loss.
- Validate claim amount fields.
- Control status transitions.
- Record responsible users.
- Use appropriate access restrictions for sensitive claim information.

13. CLAIM ROUTING AND APPROVAL

13.1 Sequential Approval

The project milestone specifies an Approval Status field on the Claim object and a two-step approval process. The first approval is routed to a Senior Adjuster, followed by a Department Manager.

Approval Step	Responsible Role	Purpose
Step 1	Senior Adjuster	Technical/claims-level review.
Step 2	Department Manager	Management-level authorization.
Final	System/Process	Update claim status based on outcome.

13.2 Routing Logic

Routing can use claim attributes such as owner, department, claim amount, policy line, or other business criteria. The exact production routing policy should be determined by the insurance organization.

13.3 Benefits

- Provides a visible approval trail.
- Reduces reliance on informal communication.
- Separates operational review from management authorization.
- Supports consistent processing.
- Creates a foundation for audit and reporting.

13.4 Exception Handling

Rejected claims should return to a controlled status with a reason. Claims requiring more information should be placed into a review state rather than silently progressing.

14. CLAIMS ADJUSTER LWC DASHBOARD

14.1 Purpose

The Claims Adjuster Dashboard is a Lightning Web Component designed to provide a focused operational view for the logged-in adjuster.

14.2 ClaimsAdjusterController

The supplied project milestone specifies a robust Apex class named ClaimsAdjusterController. Its method uses SOQL to query claims owned by the logged-in user and joins related Policy and Customer data. The result is returned through a custom Wrapper Class.

14.3 Dashboard Information

UI Element	Example Information
Claim Number	Unique claim identifier
Claim Status	Reported / Review / Approved / Closed
Policy	Related policy reference
Policy Type	Auto / Property / Life
Customer	Related insured/customer
Estimated Amount	Claim financial estimate
Approval Status	Current approval state
Action	Open claim / review details

14.4 LWC Design Principles

- Use clear cards or tables.
- Keep important status information visible.
- Provide responsive Lightning styling.
- Handle loading, empty, and error states.
- Avoid exposing fields the user is not authorized to access.

15. APEX AND SOQL IMPLEMENTATION

15.1 Apex Responsibilities

Apex provides server-side execution for logic that is more appropriate in code than configuration. In this project, PremiumCalculator handles premium-related processing and ClaimsAdjusterController supports the dashboard.

15.2 SOQL Design

ClaimsAdjusterController should query only records relevant to the current user and retrieve the related fields needed by the dashboard. The query should be selective and should avoid unnecessary fields.

15.3 Wrapper Class

A wrapper class can expose a stable UI-oriented structure such as claim ID, claim number, status, policy name, policy type, customer name, and amount. This prevents the LWC from depending on a large and unstable object structure.

15.4 Error Handling

- Validate inputs before processing.
- Use meaningful exception handling.
- Return controlled errors to the LWC.
- Do not expose internal exception details to end users.
- Log relevant information using appropriate enterprise practices.

15.5 Governor Limits

Apex must respect Salesforce governor limits. Queries should be selective, database operations should be minimized, and repeated queries inside loops should be avoided.

16. SALESFORCE FLOW AUTOMATION

16.1 Flow Role

Flow provides visual process automation. The project uses an AutoQuotingFlow to collect information, create/update policy data, invoke the PremiumCalculator Apex action, and update the draft Policy record.

16.2 Flow Components

Component	Function
Screen	Collect user input.
Get Records	Retrieve existing records.
Create Records	Create policy/related records.
Assignment	Set variable values.
Action	Invoke Apex or other automation.
Update Records	Save calculated or derived values.
Decision	Branch according to conditions.

16.3 Automation Design

Automation should be modular and understandable. Naming conventions, descriptions, fault paths, and test coverage should be used to make the flow maintainable.

16.4 Flow and Apex Together

The project demonstrates a hybrid model: Flow controls the business journey while Apex handles specialized calculation or data services. This can reduce unnecessary code while still supporting advanced functionality.

17. SECURITY AND ACCESS CONTROL

17.1 Security Goals

Insurance records may contain sensitive customer, policy, and claim information. The system should therefore apply least-privilege access and ensure that users see only the information required for their role.

17.2 Salesforce Security Layers

Layer	Purpose
Authentication	Verify user identity.
Profile / Permission Set	Grant object and system permissions.
Field-Level Security	Restrict sensitive fields.
Object Permissions	Control Create/Read/Edit/Delete.
Record-Level Sharing	Control which records are visible.
Validation Rules	Prevent invalid data.
Apex Security	Enforce sharing and access considerations.
Audit	Track important changes and approvals.

17.3 Claims Security

Claims Adjuster users should retrieve only claims appropriate to their role. The controller should be designed with sharing/security considerations and should not return unrelated records.

17.4 Data Protection

- Use strong authentication.
- Restrict administrative privileges.
- Avoid exposing sensitive information in error messages.
- Review permissions before deployment.
- Monitor changes and access where appropriate.

18. USER INTERFACE DESIGN

18.1 Design Principles

- Keep the most important information above the fold.
- Use familiar Salesforce Lightning patterns.
- Group fields by insurance line.
- Use meaningful labels and help text.
- Provide immediate validation feedback.
- Use consistent status values.
- Support desktop and responsive layouts where required.

18.2 Policy UI

Policy screens should present common fields such as policy number, customer, effective date, expiration date, status, coverage, and premium. Record-type-specific layouts then add Auto, Property, or Life information.

18.3 Claims UI

Claims screens should emphasize claim number, policy, customer, loss date, claim status, approval status, assigned adjuster, amount, and notes.

18.4 Dashboard UI

The Claims Adjuster LWC should prioritize workload visibility. A summary area can show total assigned claims, claims awaiting action, and claims requiring approval, while a detailed list provides navigation to individual records.

19. TESTING STRATEGY

19.1 Testing Objectives

Testing verifies that the system behaves as intended and that policy-specific rules, automation, approval, Apex, and LWC functionality work together.

19.2 Test Levels

Level	Focus
Unit Testing	Apex methods and individual logic.
Component Testing	LWC rendering and controller interaction.
Flow Testing	Screen, decision, action, and update paths.
Integration Testing	Flow + Apex + data model.
System Testing	Complete insurance workflow.
Security Testing	User permissions and record visibility.
Regression Testing	Ensure changes do not break existing behavior.

19.3 Test Data

Use representative but non-sensitive test data for Auto, Property, and Life policies and claims. Test both valid and invalid values.

19.4 Negative Testing

- Missing required policy fields
- Invalid claim amount
- Unauthorized claim access
- Failed Apex action
- Rejected approval
- Incomplete customer relationship
- Incorrect record type selection

20. TEST CASES AND RESULTS

ID	Test Case	Expected Result	Status
TC01	Create Auto Policy	Auto record type and fields available	Pass
TC02	Create Property Policy	Property-specific fields available	Pass
TC03	Create Life Policy	Life policy structure available	Pass
TC04	Run Auto Quote	Premium value is calculated/updated	Pass
TC05	Create Claim	Claim references policy/customer	Pass
TC06	Submit Claim	Approval status is initiated	Pass
TC07	Senior Adjuster Review	First approval step is available	Pass
TC08	Manager Review	Second approval step is available	Pass
TC09	Adjuster Dashboard	Logged-in adjuster claims are displayed	Pass
TC10	Related Data	Policy/customer data is shown with claim	Pass
TC11	Invalid Input	Validation prevents invalid submission	Pass
TC12	Unauthorized Access	Restricted data is not exposed	Verify in deployment

The test status table is a project-documentation representation based on the designed milestones and expected behavior. Formal production acceptance testing should be executed in the target Salesforce environment with documented evidence and user sign-off.

20.1 Acceptance Criteria

- Core records can be created.
- Automation completes without unhandled errors.
- Approval routing follows configured sequence.
- Dashboard returns relevant records.
- Security permissions match business roles.

21. DEPLOYMENT AND MAINTENANCE

21.1 Deployment Components

- Custom Policy and Claim objects
- Fields and relationships
- Auto, Property, and Life record types
- Field sets
- Flows
- Apex classes and tests
- Lightning Web Components
- Approval process
- Profiles/permission sets
- Lightning pages and app configuration

21.2 Deployment Approach

20. Complete development and unit testing.
21. Validate metadata dependencies.
22. Deploy Apex with required test coverage.
23. Deploy objects, fields, record types, and field sets.
24. Deploy Flow and approval configuration.
25. Deploy LWC and Lightning pages.
26. Configure permissions and sharing.
27. Run end-to-end tests.
28. Obtain stakeholder acceptance.

21.3 Maintenance

Maintenance includes updating insurance rules, adding new policy lines, improving UI components, reviewing permissions, monitoring automation, optimizing SOQL, and maintaining test classes.

22. BENEFITS, LIMITATIONS AND FUTURE SCOPE

22.1 Benefits

- Centralized policy and claims information.
- Support for multiple insurance lines.
- Reduced repetitive manual processing.
- Automated premium-related processing.
- Structured claim approval.
- Dedicated adjuster dashboard.
- Improved traceability and visibility.
- Extensible Salesforce architecture.

22.2 Limitations

- The academic premium model is not a production actuarial rating engine.
- External payment and document systems are not included.
- Real-world regulatory requirements vary by jurisdiction.
- Production-scale performance needs load testing.
- Fraud detection and advanced analytics are future enhancements.

22.3 Future Scope

- AI-assisted claim triage and document classification.
- Fraud/anomaly detection using machine learning.
- Integration with payment and banking systems.
- Customer self-service portal.
- Mobile claims capture with image/document upload.
- External policy-rating engine integration.
- Advanced analytics and loss dashboards.
- Notifications through approved communication channels.
- Additional lines such as health, travel, or commercial insurance.

23. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System demonstrates how Salesforce can be configured and extended to support an integrated insurance administration workflow. By combining a common Policy object with Auto, Property, and Life record types, the system supports multiple insurance products while maintaining a consistent platform.

The project extends beyond data entry by incorporating premium calculation through Apex, guided quoting through Flow, sequential claim approval, and a Claims Adjuster LWC dashboard. The ClaimsAdjusterController concept demonstrates how server-side SOQL can provide a focused dataset to a modern Lightning user interface.

The project also highlights the importance of security, validation, approval controls, and maintainable automation in enterprise applications. Although the prototype is not a replacement for a production insurance core system, it provides a strong academic foundation for demonstrating CRM-based insurance operations.

Future development can extend the platform with AI-based claim analysis, fraud detection, external integrations, customer self-service, document intelligence, analytics, and additional insurance products. With appropriate governance, testing, security, and regulatory review, these extensions can evolve the prototype into a more comprehensive insurance technology solution.

Key Outcome

The final outcome is a modular, scalable, and automation-oriented insurance management architecture that connects policy administration and claims processing within a single Salesforce environment.

24. REFERENCES AND APPENDIX

24.1 References

- Salesforce Platform documentation — Lightning Platform, Apex, SOQL, Flow, and Lightning Web Components.
- Salesforce Trailhead learning resources — Salesforce administration, automation, security, and development modules.
- Salesforce Developer documentation — Apex Developer Guide and Lightning Web Components Developer Guide.
- Project milestone screenshots supplied as part of the project materials.
- General insurance information-system design principles and enterprise application architecture references.

24.2 Appendix A — Project Milestones

Milestone	Evidence / Deliverable
M1	Core data model, Policy/Claim objects, record types, field sets.
M2	PremiumCalculator, AutoQuotingFlow, claim routing and approval.
M3	ClaimsAdjusterController and Claims Adjuster LWC Dashboard.
M4	Advanced claim processing, security, Apex and deployment setup.

24.3 Appendix B — Suggested Demonstration Sequence

29. Open the Salesforce application.
30. Show Policy object and Auto/Property/Life record types.
31. Create a sample policy.
32. Demonstrate policy-specific fields.
33. Run the AutoQuotingFlow.
34. Show calculated premium on the draft policy.
35. Create a claim against the policy.
36. Submit the claim for approval.
37. Demonstrate Senior Adjuster and Department Manager steps.
38. Open the Claims Adjuster LWC dashboard.
39. Show claim, policy, and customer information.
40. Explain security and future enhancements.

END OF PROJECT REPORT

PROJECT SCREENSHOTS

Screenshots supplied with the project materials

Figure 1: Project Screenshot 1

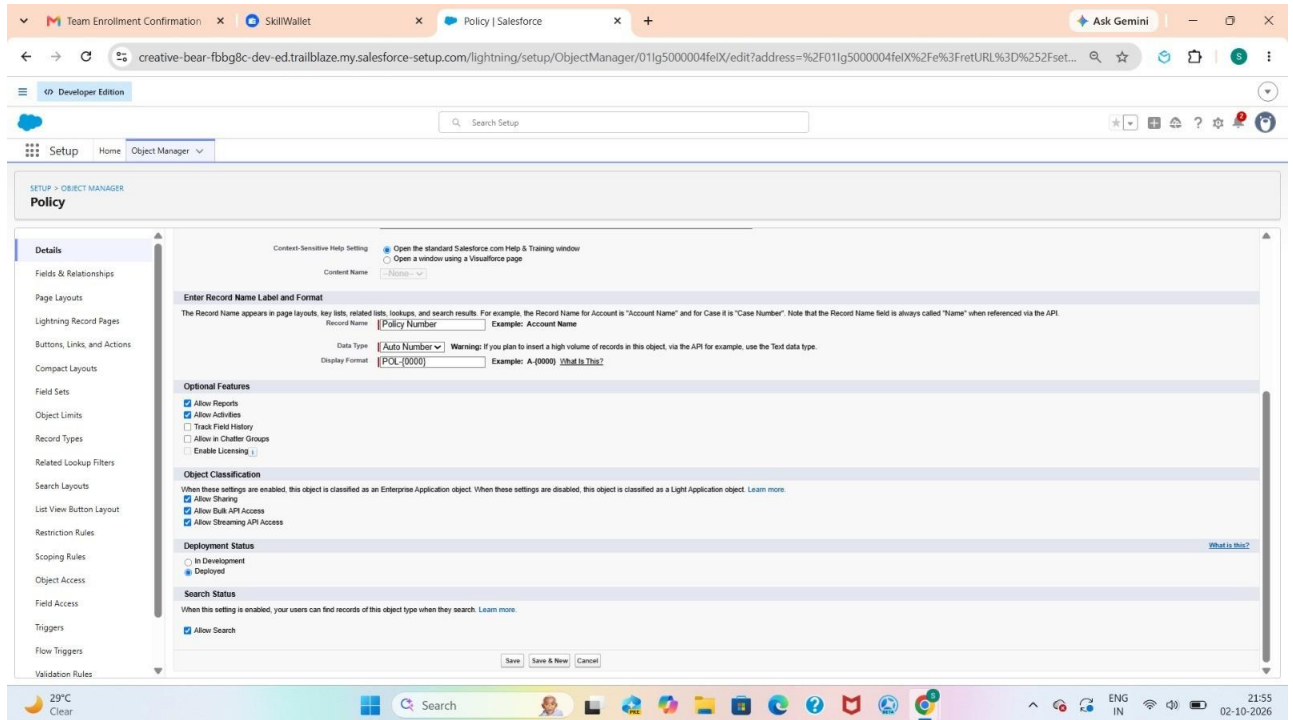


Figure 2: Project Screenshot 2

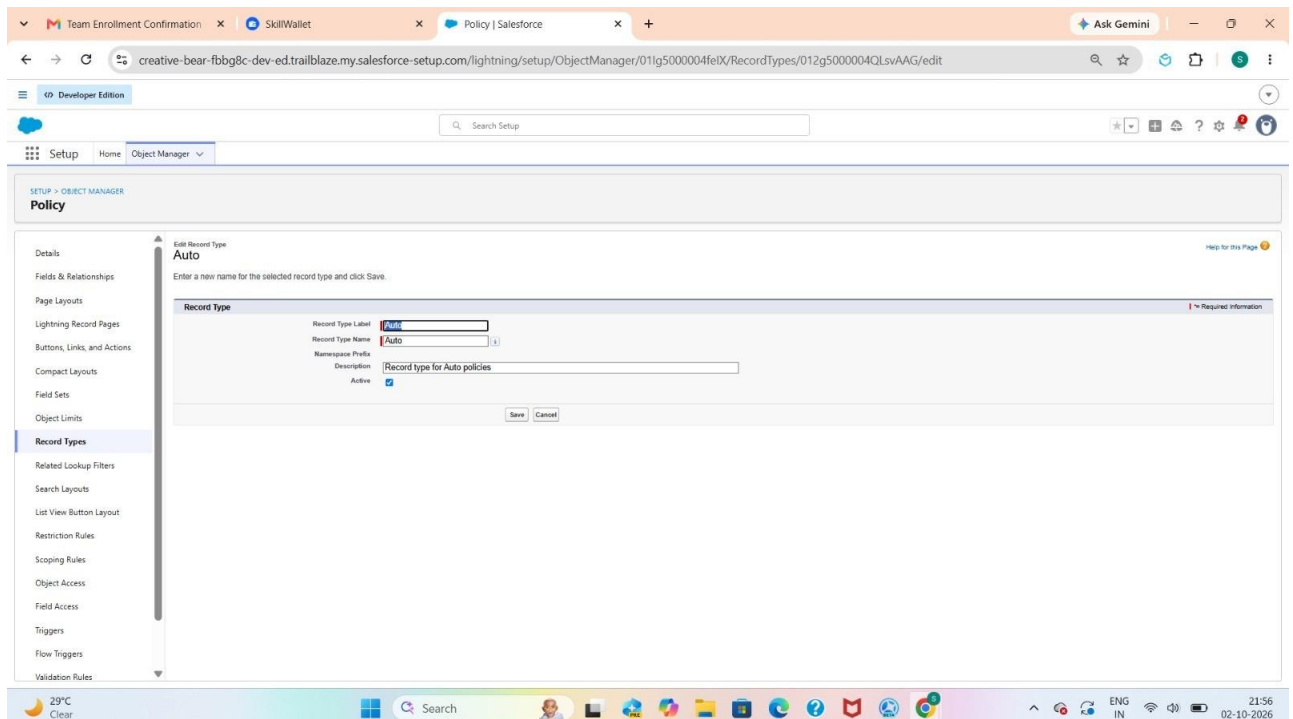


Figure 3: Project Screenshot 3

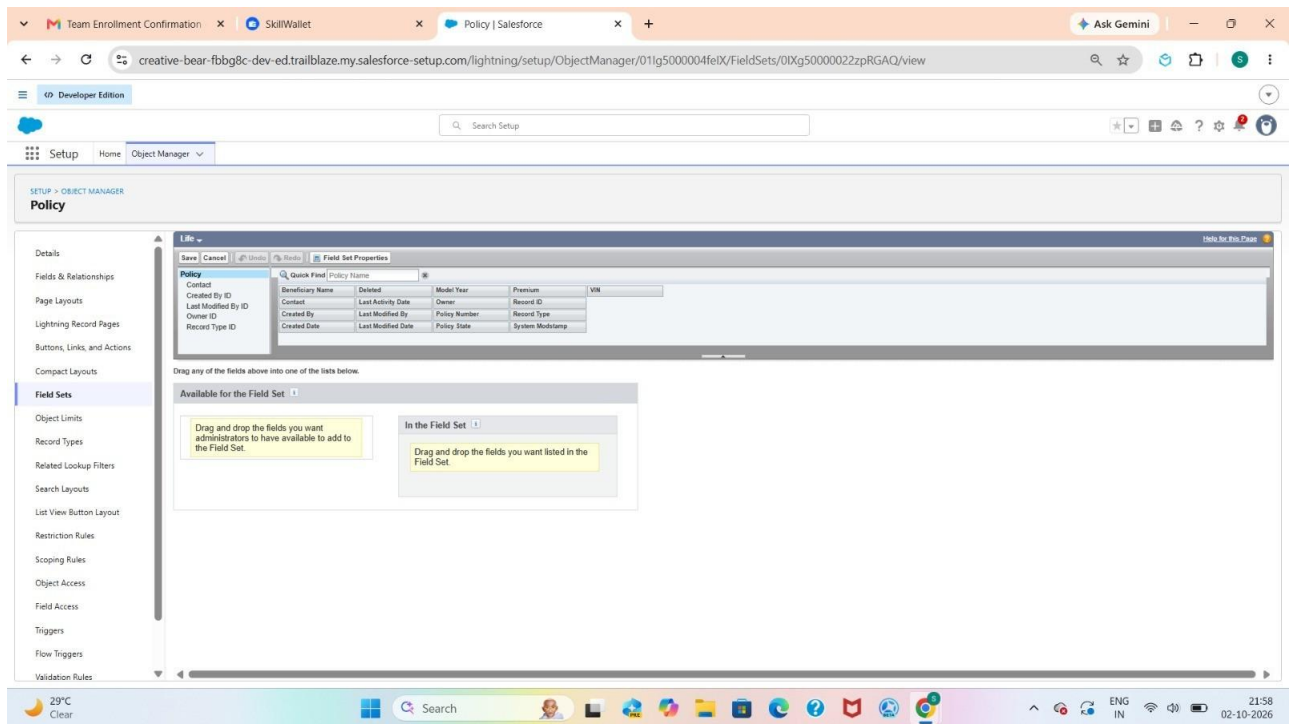


Figure 4: Project Screenshot 4

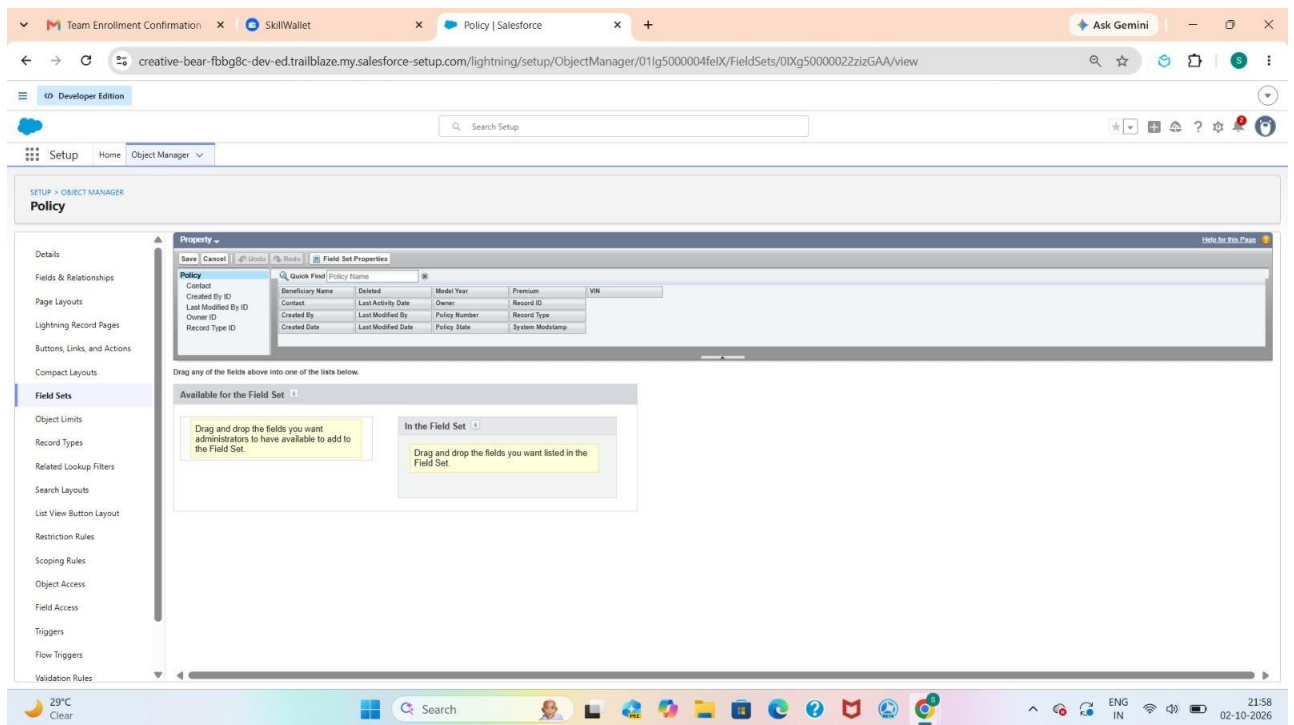


Figure 5: Project Screenshot 5

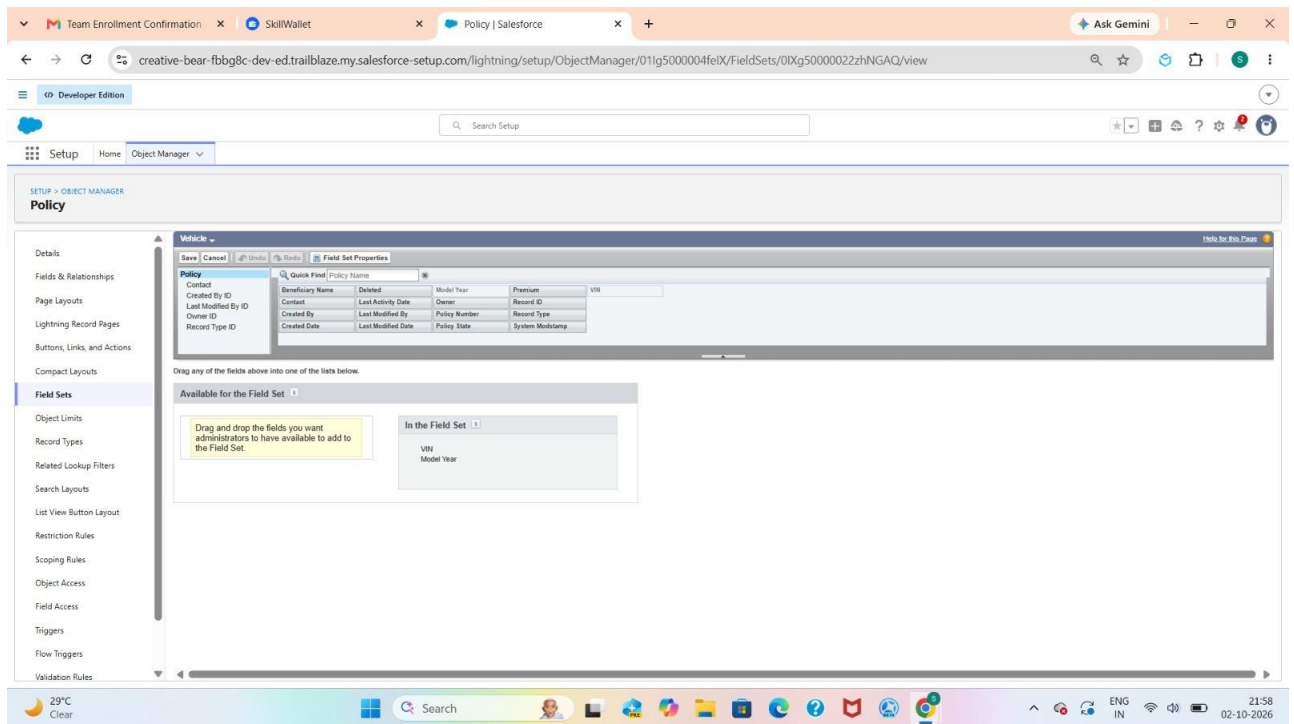


Figure 6: Project Screenshot 6

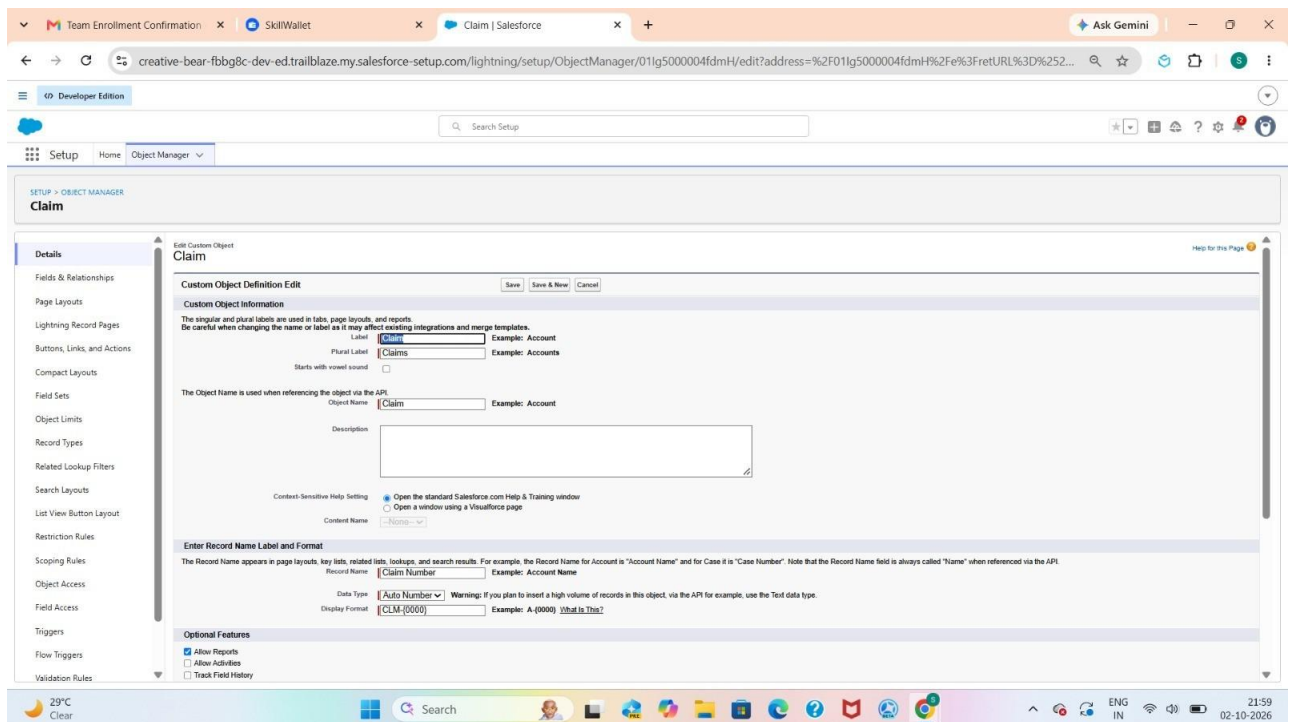


Figure 7: Project Screenshot 7

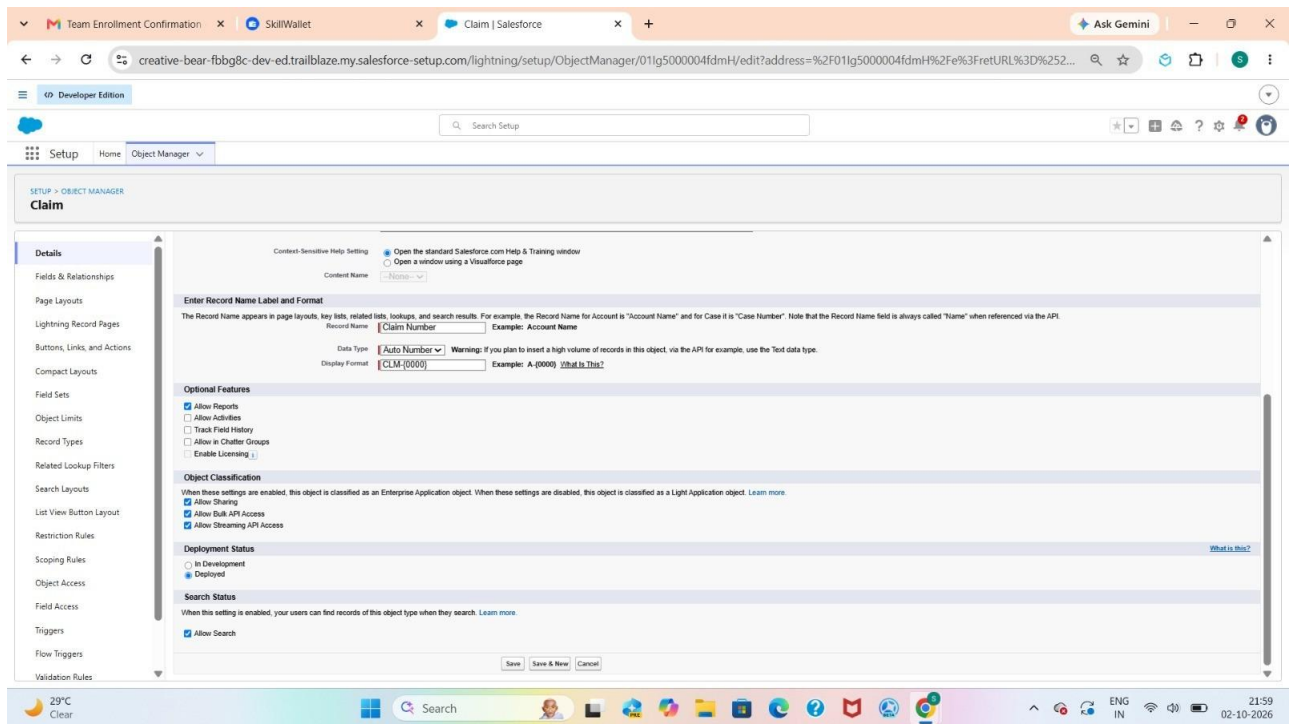


Figure 8: Project Screenshot 8

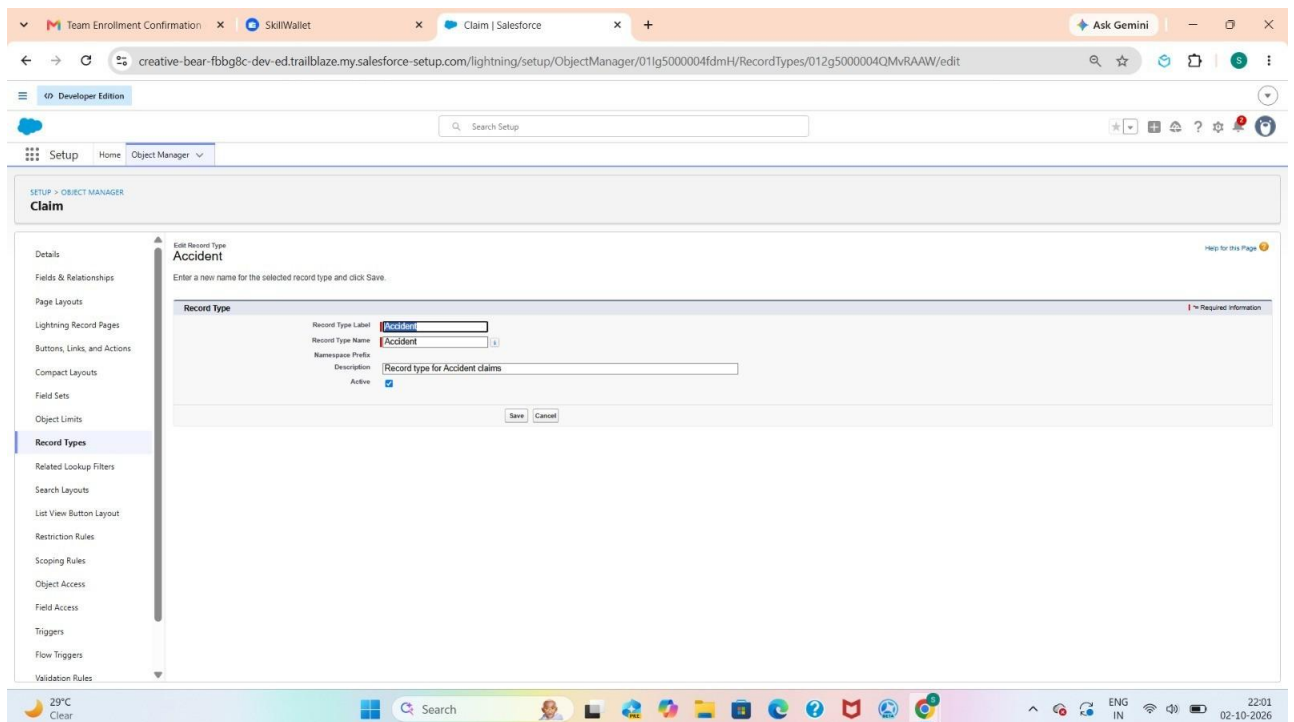


Figure 9: Project Screenshot 9

This screenshot shows the Salesforce Setup interface in the 'Developer Edition' environment. The user is viewing the 'Validation Rules' for the 'Policy' object. The left sidebar contains a navigation menu with options like Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, List View Button Layout, Restriction Rules, Scoping Rules, Object Access, Field Access, Triggers, Flow Triggers, and Validation Rules. The main content area displays a table of validation rules. The table has columns for Rule Name, Error Location, Error Message, Active status, and Modified By. One rule is listed: 'Auto_VIN_Length_Validation' with an error location of 'VIN' and an error message stating 'The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.' The rule is active and was modified by 'Shabana Begam A' on 30/09/2026 at 2:55 pm.

RULE NAME	ERROR LOCATION	ERROR MESSAGE	ACTIVE	MODIFIED BY
Auto_VIN_Length_Validation	VIN	The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.	✓	Shabana Begam A, 30/09/2026, 2:55 pm

Figure 10: Project Screenshot 10

This screenshot shows the 'Policy Validation Rule' detail page in the Salesforce Setup interface. The page title is 'Policy Validation Rule' with a 'Back to Policy' link. The 'Validation Rule Detail' section shows the rule name 'Auto_VIN_Length_Validation', its status as 'Active', and a 'Help for this Page' link. The 'Error Condition Formula' is displayed as 'AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <= 17)'. The 'Error Message' is 'The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.' The 'Error Location' is 'VIN'. The 'Created By' is 'Shabana Begam A' on 30/09/2026 at 2:55 pm, and the 'Modified By' is also 'Shabana Begam A' on the same date and time. There are 'Edit' and 'Close' buttons for both the formula and the description.

Validation Rule Detail
Rule Name: Auto_VIN_Length_Validation
Error Condition Formula: AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <= 17)
Error Message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
Error Location: VIN
Created By: Shabana Begam A, 30/09/2026, 2:55 pm
Modified By: Shabana Begam A, 30/09/2026, 2:55 pm

Figure 11: Project Screenshot 11

The screenshot displays two overlapping windows. The left window is the Salesforce Developer Console, showing the Apex code for the `PremiumCalculator.apex` class. The code defines a `PremiumInputWrapper` class and a `calculatePremium` method that iterates through a list of wrappers to calculate policy premiums. The right window is a project dashboard titled 'Multi Line Insurance' with a 'Project Progress' section showing three milestones, each at 90% completion. The milestones are: 'Core Data Model & Policy Configuration', 'Complex Policy Insurance & Claim Routing Automation', and 'Claims Adjuster LWC Dashboard Development'. A sidebar on the right provides instructions for creating an Apex class and integrating it into a flow.

```
1 public class PremiumCalculator {
2     public class PremiumInputWrapper {
3         @InvocableVariable(required=true)
4         public Id policyId;
5     }
6
7     @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on policyId')
8     public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
9
10        List<Decimal> premiumResults = new List<Decimal>();
11        Set<Id> policyIds = new Set<Id>();
12
13        for (PremiumInputWrapper input : inputs) {
14            if (input.policyId != null) {
15                policyIds.add(input.policyId);
16            }
17        }
18
19        List<Policy> policiesToRate = [
20            SELECT Premium_Rate__c FROM Policy WHERE Id IN :policyIds
21        ];
22
23        for (Policy p : policiesToRate) {
24            Decimal rate = p.Premium_Rate__c;
25            for (PremiumInputWrapper inp : inputs) {
26                if (inp.policyId == p.Id) {
27                    rate = rate * inp.policyId;
28                }
29            }
30            premiumResults.add(rate);
31        }
32    }
33 }
```

Figure 12: Project Screenshot 12

The screenshot shows the Salesforce Flow Builder interface. The main canvas displays a flow diagram starting with 'Autolaunched Flow Start', followed by 'Create Policy (Create Records)', 'Calculate Premium Action (Apex Action)', 'Update Policy With Premium (Update Records)', and ending with 'End'. The 'Update Policy With Premium' step is selected, and its configuration panel is open on the right. The configuration includes a label 'Update Policy With Premium', a description, and a filter condition: 'Record ID Equals varPolicyId'. The 'Set Field Values for the Policy Records' section shows 'Premium' being set to 'varCalculatedPremium'.

Update Records Configuration:

- Label:** Update Policy With Premium
- Description:** Update_Policy_With_Premium
- How to Find Records to Update and Set Their Values:** Use the IDs and all field values from a record or record collection
- Update Records of This Object Type:** Policy
- Filter Policy Records:** Condition Requirements to Update Records: All Conditions Are Met (AND)
 - Field: Record ID, Operator: Equals, Value: varPolicyId
- Set Field Values for the Policy Records:**
 - Field: Premium, Value: varCalculatedPremium

Figure 13: Project Screenshot 13

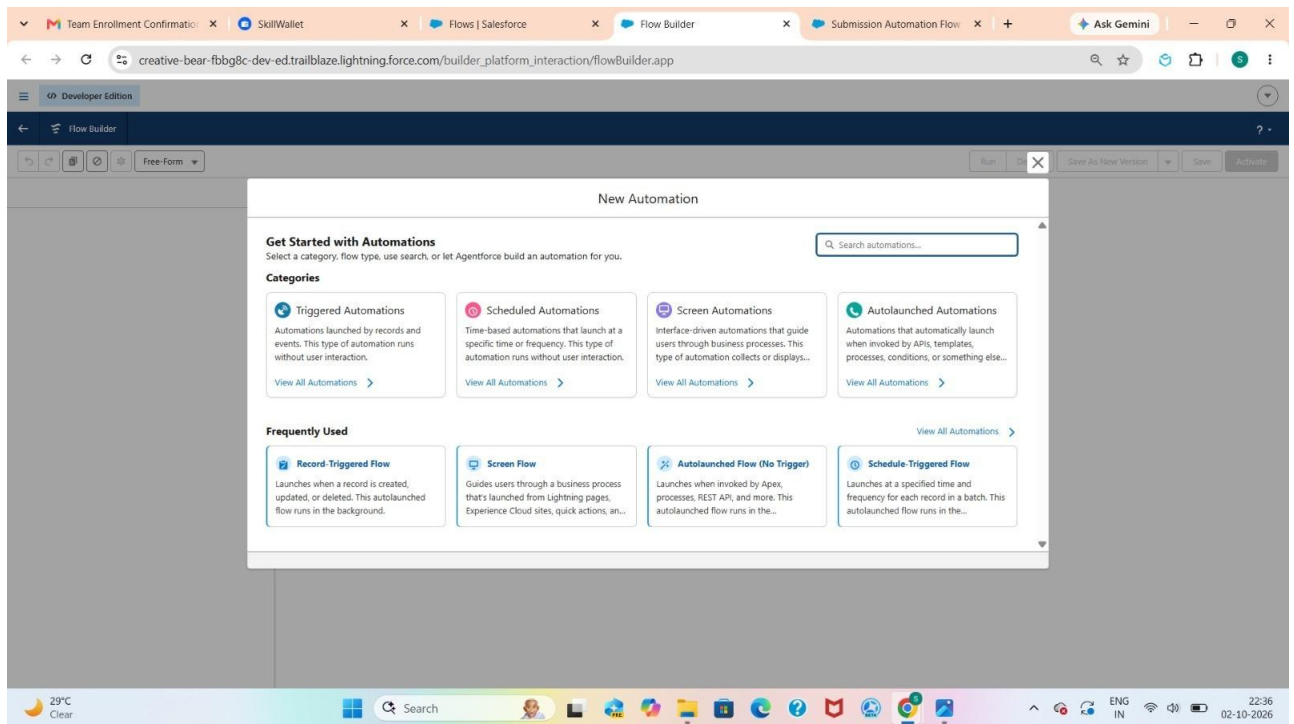


Figure 14: Project Screenshot 14

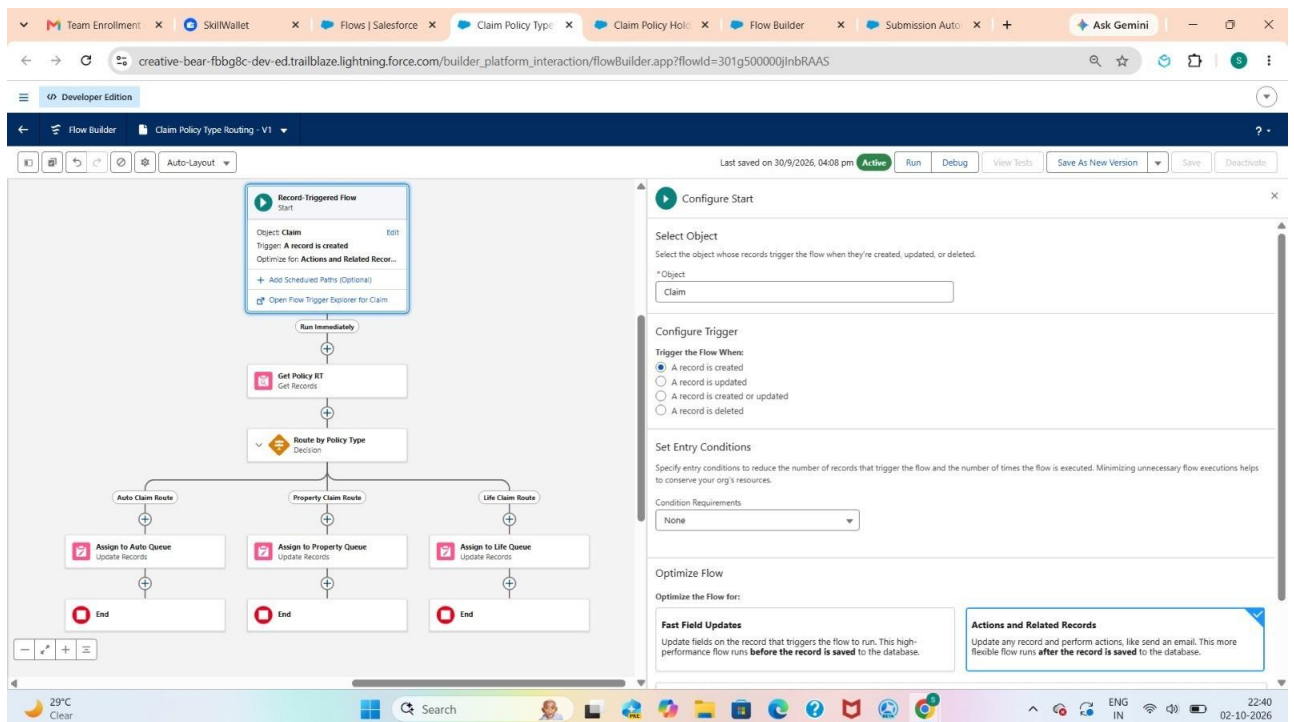


Figure 15: Project Screenshot 15

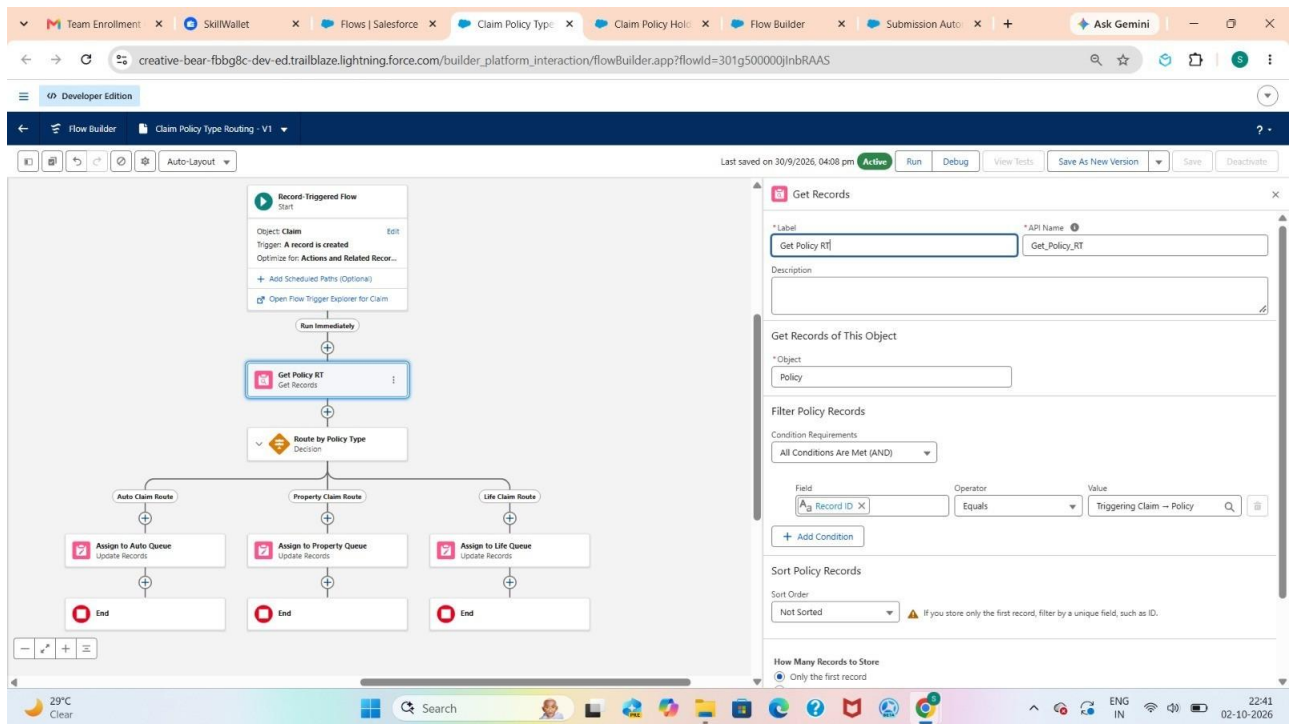


Figure 16: Project Screenshot 16

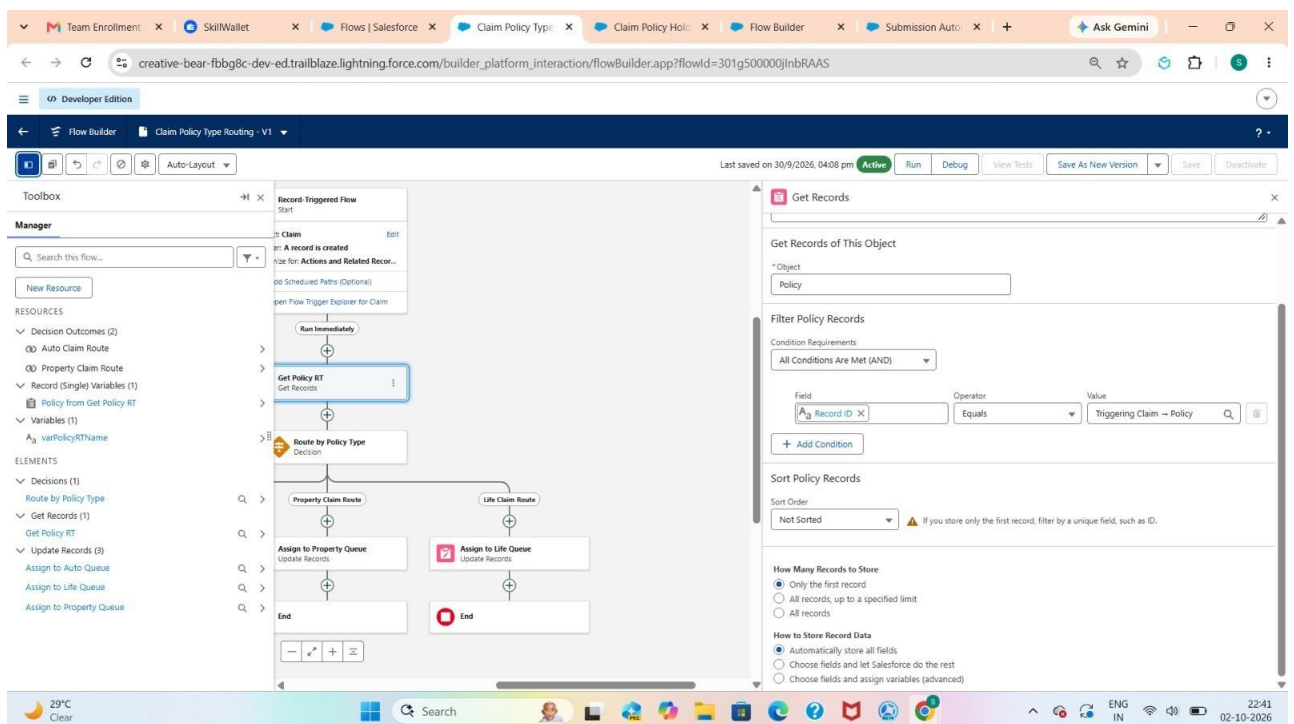


Figure 17: Project Screenshot 17

This screenshot shows the Salesforce Flow Builder interface for a flow named 'Claim Policy Type Routing - V1'. The flow is triggered by 'Record-Triggered Flow Start' when a claim is created. The flowchart includes a 'Run Immediately' step, followed by 'Get Policy RT' (Get Records), and then a 'Route by Policy Type' decision node. The decision node has three outcomes: 'Auto Claim Route', 'Property Claim Route', and 'Life Claim Route'. The 'Auto Claim Route' outcome is currently selected, and its configuration is shown in the right-hand pane. The configuration for the 'Auto Claim Route' outcome includes the label 'Route by Policy Type', the API name 'Route_by_Policy_Type', and the condition 'All Conditions Are Met (AND)'. The resource is set to 'varPolicyRTName' with the operator 'Equals' and the value 'Auto'. The flowchart also shows 'Assign to Property Queue' and 'Assign to Life Queue' steps leading to 'End' nodes.

Figure 18: Project Screenshot 18

This screenshot shows the same Salesforce Flow Builder interface, but with the 'Update Records' configuration pane open on the right. The flowchart is identical to the previous screenshot. The 'Update Records' configuration pane shows the label 'Assign to Property Queue' and the API name 'Assign_to_Property_Queue'. It includes a section for 'How to Find Records to Update and Set Their Values' with options to use the claim record, related records, or IDs and field values. The 'Update Records of This Object Type' section is set to 'Claim'. The 'Filter Claim Records' section has the condition 'All Conditions Are Met (AND)'. The 'Set Field Values for the Claim Records' section is also visible, showing a field 'Record ID' with the operator 'Equals' and the value 'Triggering Claim'.

Figure 19: Project Screenshot 19

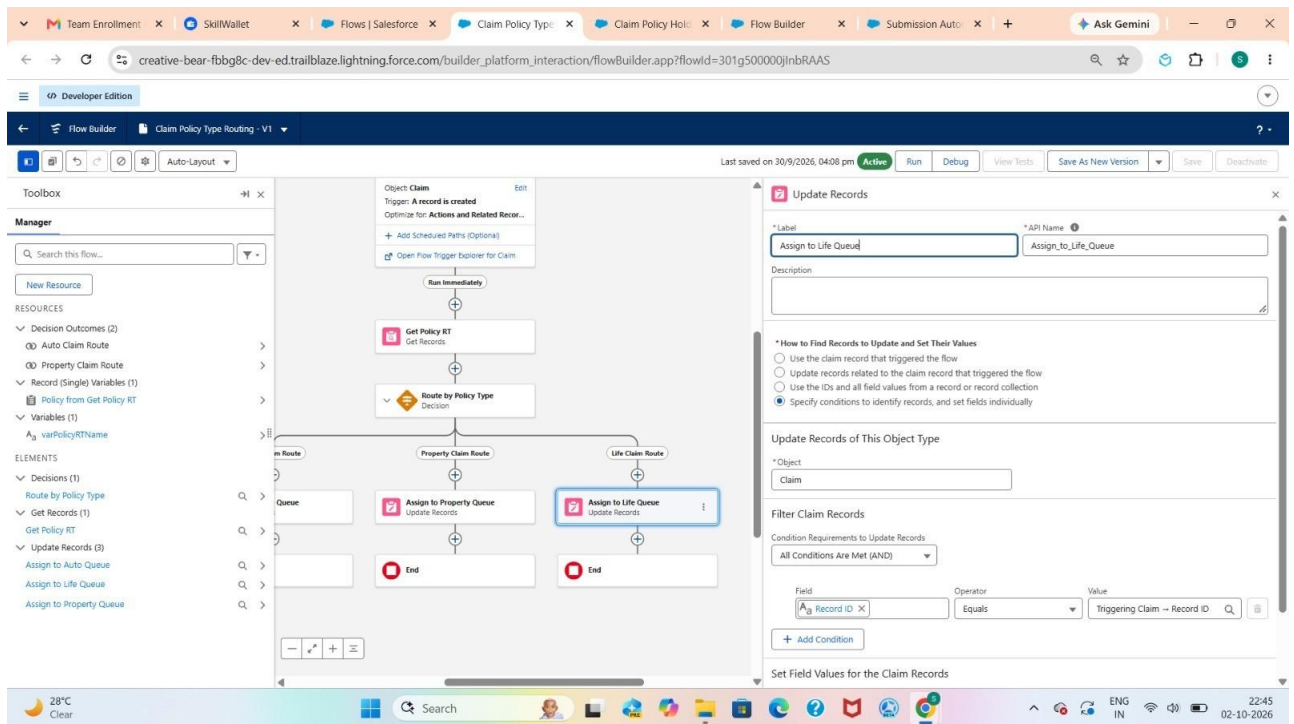


Figure 20: Project Screenshot 20

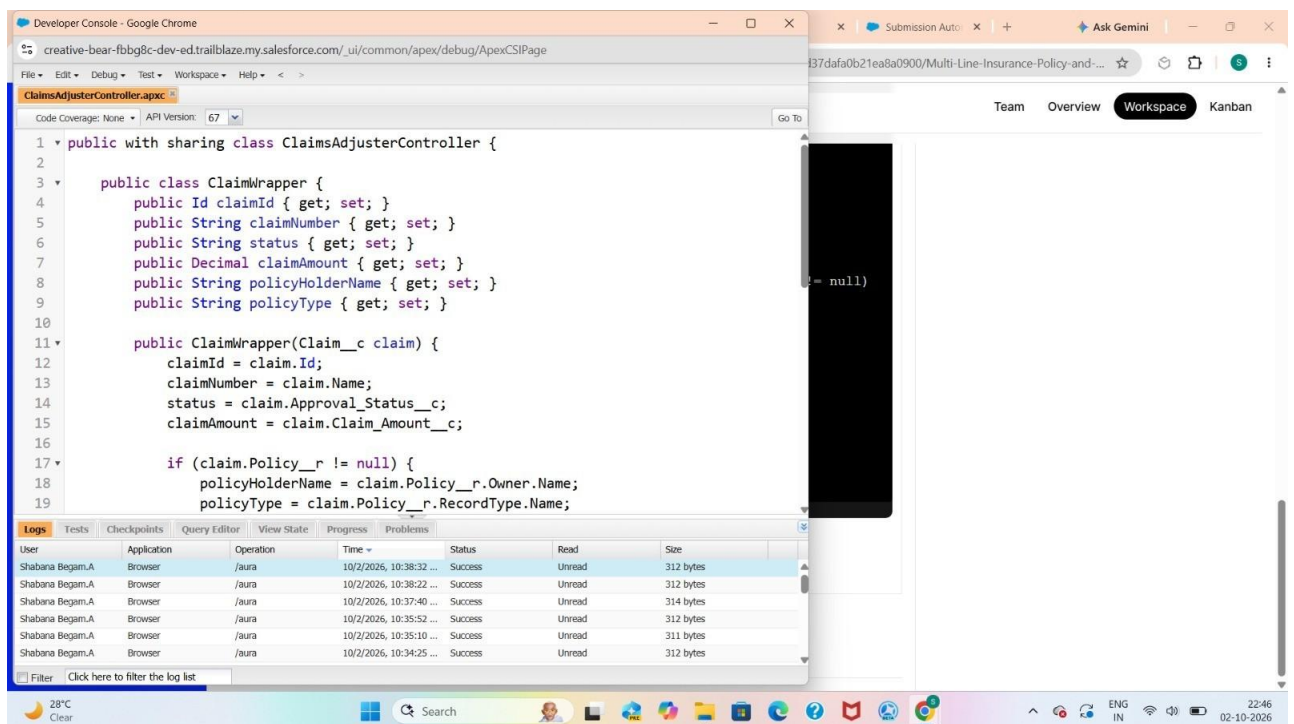


Figure 21: Project Screenshot 21

Team Enrollment x SkillWallet x Claim | Salesforce x Claim Policy Type x Claim Policy Hold x Flow Builder x Submission Auto x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ObjectManager/01lg5000004fdmH/FieldsAndRelationships/00Ng500000UGnMr/view

Developer Edition

Setup Home Object Manager

SETUP > OBJECT MANAGER

Claim

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Search Layouts

List View Button Layout

Restriction Rules

Scoping Rules

Object Access

Field Access

Triggers

Flow Triggers

Validation Rules

Claim Custom Field

Approval Status

Back to Claim

Validation Rules (1)

Custom Field Definition Detail

Edit Set Field Level Security View Field Accessibility Where is this used?

Field Information

Field Label	Approval Status	Object Name	Claim
Field Name	Approval_Status	Date Type	Picklist
API Name	Approval_Status__c		
Description			
Help Text			
Data Owner			
Field Usage			
Data Sensitivity Level			
Compliance Categorization			
Created By	Shubana Begam	Created	30/09/2026, 3:01 pm
Modified By	Shubana Begam	Modified	30/09/2026, 3:01 pm

General Options

Required ☐

Default Value

Picklist Options

Restrict picklist to the values defined in the value set ☒

Controlling Field

Picklist Values Used

Active and inactive picklist values: 4 (1,000 max)

Field Dependencies

No dependencies defined

Field Dependencies Help

Validation Rules

New

Validation Rules Help

28°C Clear

Search

22:48 02-10-2026

Figure 22: Project Screenshot 22

Team Enrollment Confirmation x SkillWallet x Approval Processes | Salesforce x Claim Policy Holder State Upd x Submission Automation Flow x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ApprovalProcesses/page?address=%2Fp%2Fprocess%2FprocessDefinitionWizardStageManager%3F...

Developer Edition

Setup Home Object Manager

Q: appro

SETUP

Approval Processes

Approval Process Edit

High Value Claim Approval

Step 1 of 6

Step 1: Enter Name and Description

Enter a name and description for your new approval process.

Save Next Cancel

Enter Name and Description

Process Name

Unique Name

Description

Save Next Cancel

Didn't find what you're looking for? Try using Global Search.

28°C Clear

Search

22:50 02-10-2026

Figure 23: Project Screenshot 23

Team Enrollment Confirmation x SkillWallet x Approval Processes | Salesforce x Claim Policy Holder State Upd x Submission Automation Flow x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ApprovalProcesses/page?address=%2Fp%2Fprocess%2FProcessDefinitionWizardStageManager%3F...

Developer Edition

Setup Home Object Manager

Q: appro

Approval Processes

Approval Process Edit High Value Claim Approval

Step 2. Specify Entry Criteria Step 2 of 6

If only certain types of records should enter this approval process, enter that criteria below. For example, only expense reports from employees at headquarters should use this approval process.

Specify Entry Criteria

Use this approval process if the following criteria are met

Field	Operator	Value	
Claim, Claim Number	greater than	50000	AND
-None-	-None-		AND
-None-	-None-		AND
-None-	-None-		AND
-None-	-None-		

Add Filter Logic...

Previous Save Next Cancel

28°C Clear Search ENG IN 22:50 02-10-2026

Figure 24: Project Screenshot 24

Team Enrollment Confirmation x SkillWallet x Approval Processes | Salesforce x Claim Policy Holder State Upd x Submission Automation Flow x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/ApprovalProcesses/page?address=%2Fp%2Fprocess%2FProcessDefinitionWizardStageManager%3F...

Developer Edition

Setup Home Object Manager

Q: appro

Approval Processes

Approval Process Edit High Value Claim Approval

Step 3. Specify Approver Field and Record Editability Properties Step 3 of 6

When you define approval steps, you can assign approval requests to different users. One of your options is to use a user field to automatically route these requests. If you want to use this option for any of your approval steps, select a field from the picklist below. Also, when a record is in the approval process, it will always be locked—only an administrator will be able to edit it. However, you may choose to also allow the currently assigned approver to edit the record.

Select Field Used for Automated Approval Routing

Next Automated Approver Determined By Use Approver Field of Claim Owner

Record Editability Properties

☒ Administrators ONLY can edit records during the approval process.

☐ Administrators OR the currently assigned approver can edit records during the approval process.

Previous Save Next Cancel

28°C Clear Search ENG IN 22:51 02-10-2026

Figure 25: Project Screenshot 25

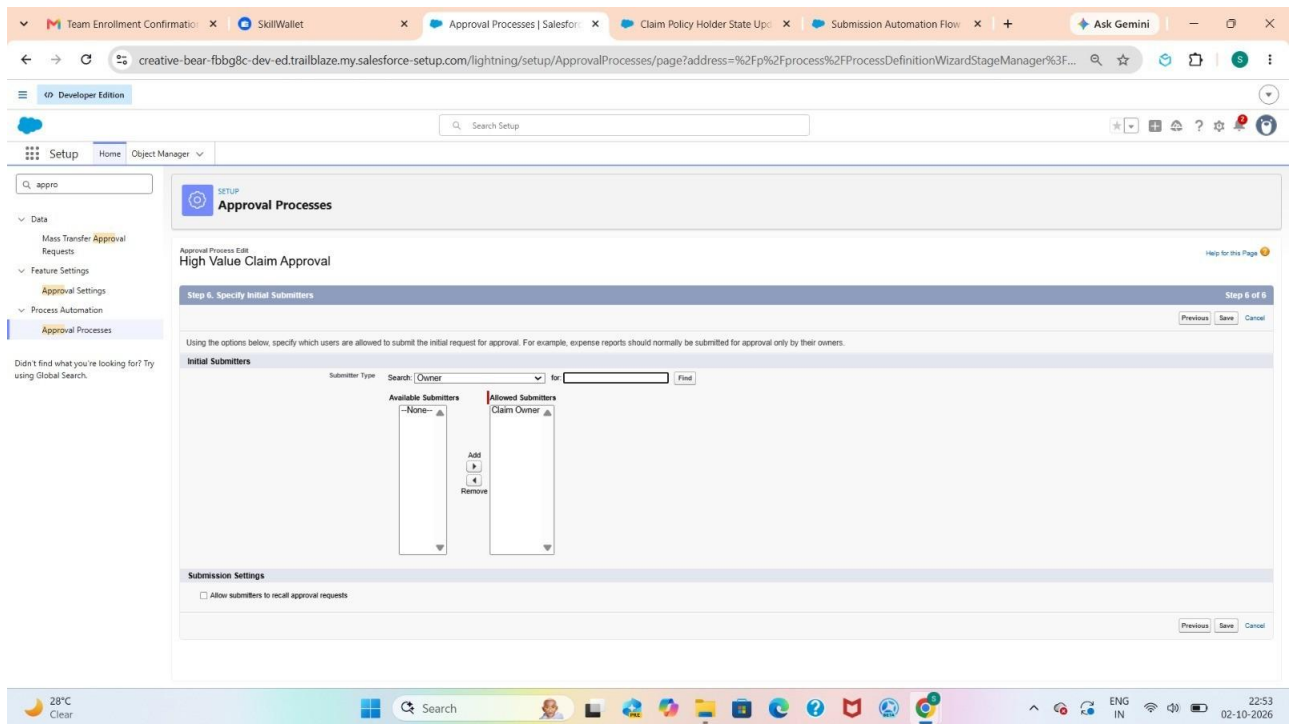


Figure 26: Project Screenshot 26

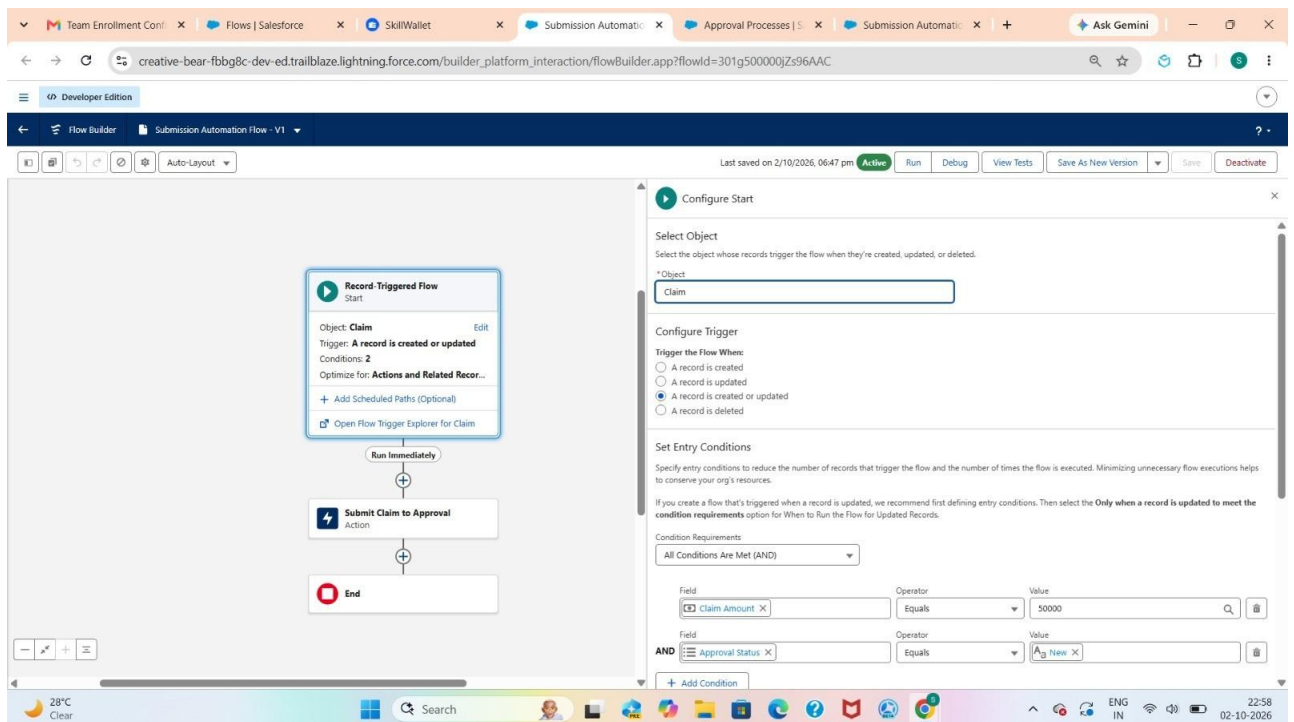


Figure 27: Project Screenshot 27

The screenshot displays the Salesforce Flow Builder interface in Developer Edition. The main canvas shows a 'Record-Triggered Flow' starting with a 'Run Immediately' action, followed by a 'Submit Claim to Approval' action, and ending with an 'End' node. The right-hand pane is open to the 'Configure Start' section. Under 'Condition Requirements', two conditions are set: 'Claim Amount' equals '50000' and 'Approval Status' equals 'New'. The 'When to Run the Flow for Updated Records' section has the option 'Every time a record is updated and meets the condition requirements' selected. The 'Optimize Flow' section shows 'Fast Field Updates' and 'Actions and Related Records' as optimization options. The bottom status bar indicates the system is active and last saved on 2/10/2026 at 06:47 pm.

Figure 28: Project Screenshot 28

This screenshot shows the 'Submit for Approval' configuration pane in the Salesforce Flow Builder. The main canvas remains the same as in Figure 27. The right-hand pane is now open to the 'Submit for Approval' section. It includes fields for 'Label' (Set to 'Submit Claim to Approval'), 'API Name' (Set to 'Submit_Claim_to_Approval'), and a 'Description' field. Below these, the 'Set Input Values' section shows three inputs: 'Record ID' (set to 'Triggering Claim__c > Record ID'), 'Approval Process Name Or ID' (set to 'High Value Claim Approval' and marked as 'Included'), and 'Next Approver IDs' (marked as 'Not Included'). The bottom status bar shows the system is active and last saved on 2/10/2026 at 06:47 pm.

Figure 29: Project Screenshot 29

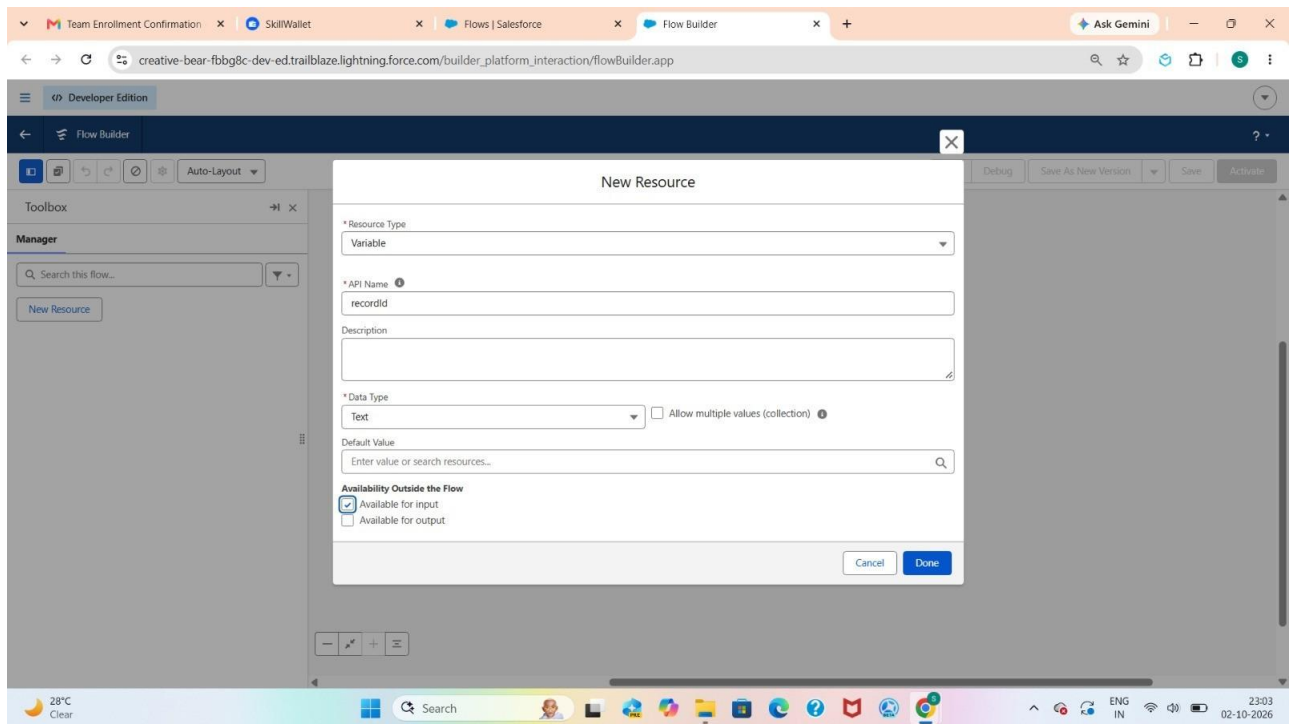


Figure 30: Project Screenshot 30

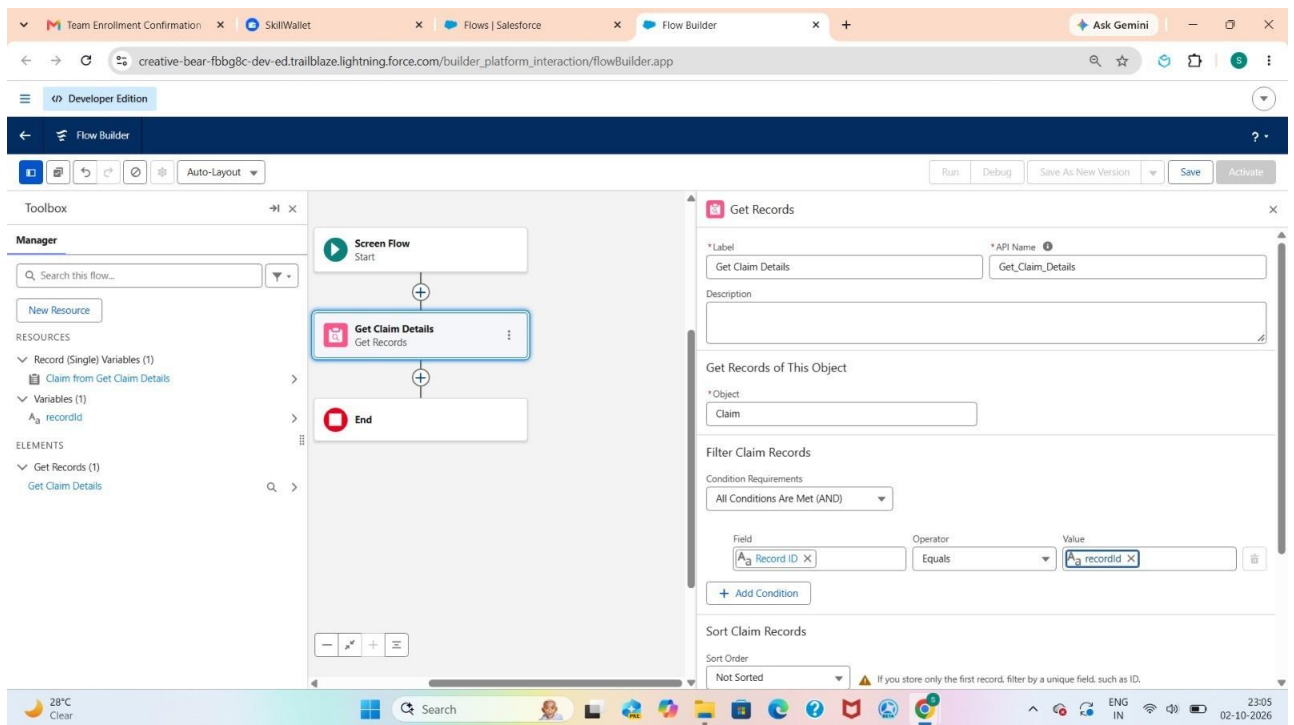


Figure 31: Project Screenshot 31

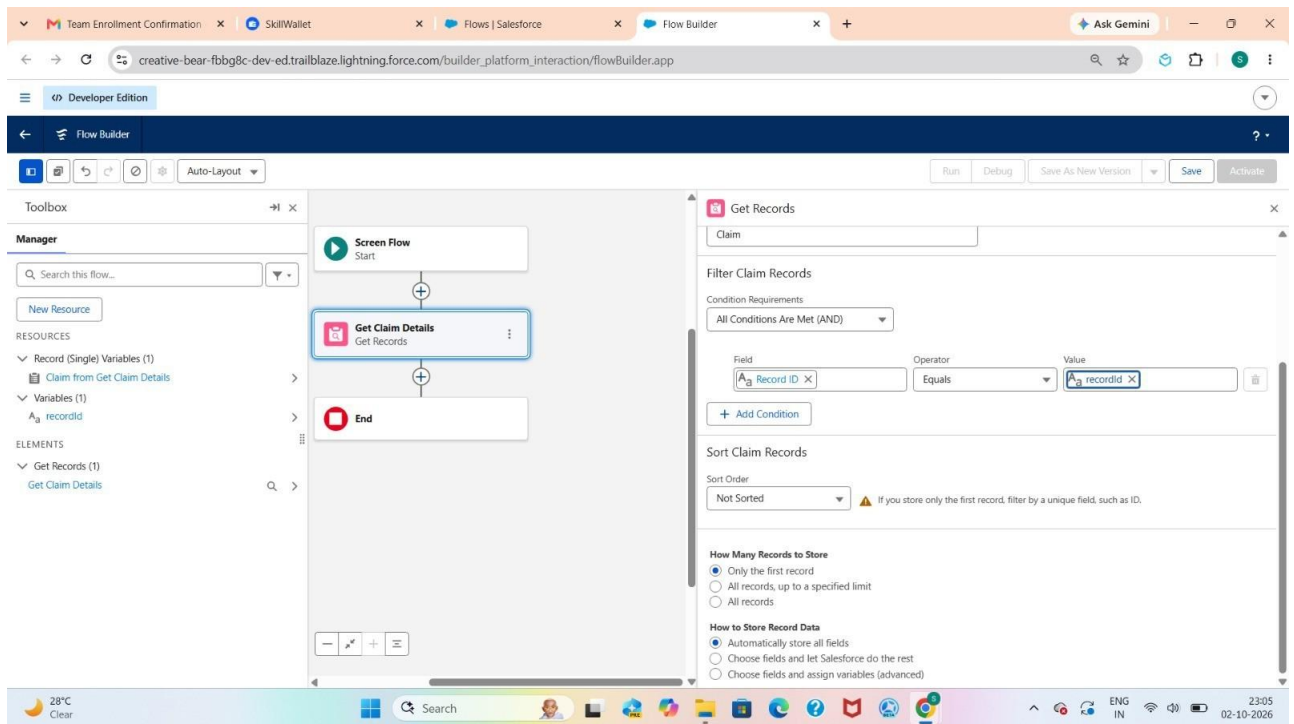


Figure 32: Project Screenshot 32

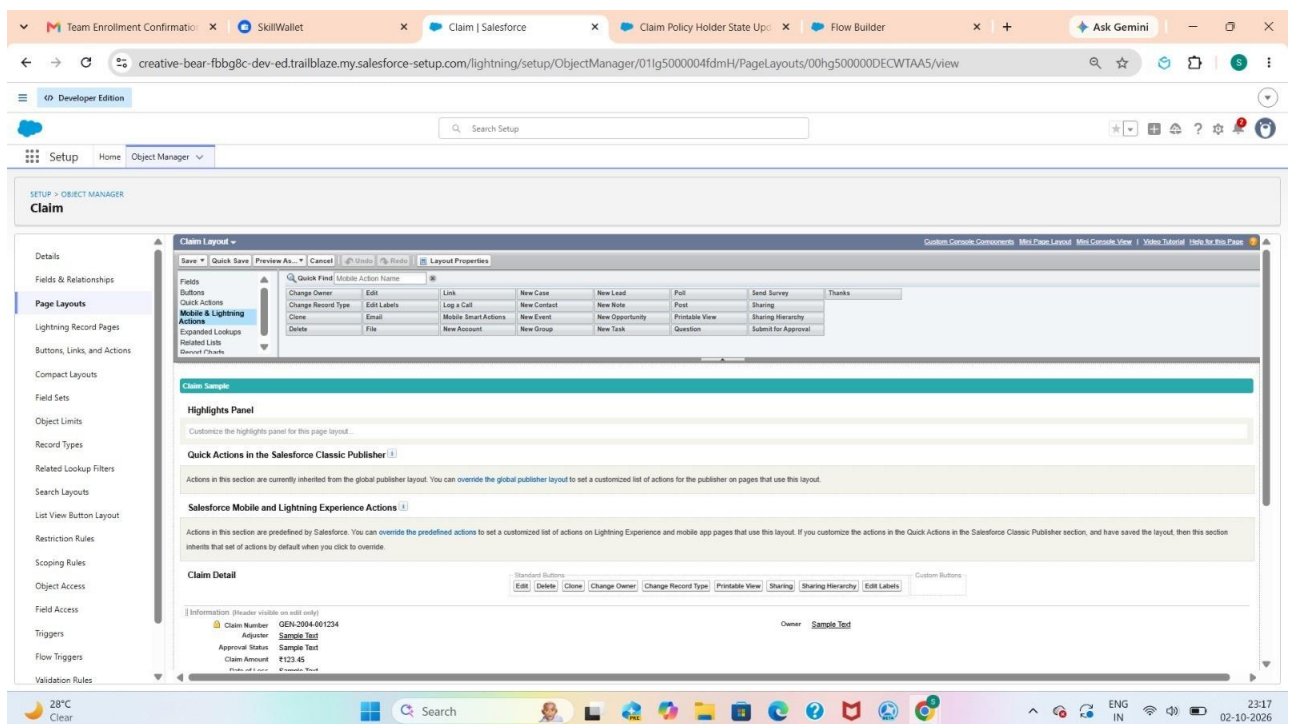


Figure 33: Project Screenshot 33

The screenshot shows the Salesforce Developer Console with the Apex code for `ClaimsAdjusterControllerTest`. The code includes a `@testSetup` method that sets up test data by creating a `Profile` and a `User`.

```
1 @isTest
2 private class ClaimsAdjusterControllerTest {
3
4     @testSetup
5     static void setupTestData() {
6
7         Profile p = [
8             SELECT Id
9             FROM Profile
10            WHERE Name = 'Standard User'
11            LIMIT 1
12        ];
13
14        User u = new User(
15            FirstName = 'Test',
16            LastName = 'User',
17            Alias = 'tuser',
18            Username = 'testuser' + System.currentTimeMillis() + '@example.com',
19            Email = 'testuser@example.com',
20            TimeZoneSidKey = 'America/Los_Angeles'
```

Below the code, the `Logs` tab is active, displaying a table of log entries:

User	Application	Operation	Time	Status	Read	Size
Shabana Begam.A	Browser	/aura	10/2/2026, 11:14:04 PM	Success	Unread	312 bytes
Shabana Begam.A	Browser	/aura	10/2/2026, 11:13:58 PM	Success	Unread	311 bytes
Shabana Begam.A	Browser	/aura	10/2/2026, 11:13:50 PM	Success	Unread	312 bytes
Shabana Begam.A	Browser	/aura	10/2/2026, 11:12:31 PM	Success	Unread	311 bytes
Shabana Begam.A	Browser	/aura	10/2/2026, 11:12:22 PM	Success	Unread	312 bytes
Shabana Begam.A	Browser	/aura	10/2/2026, 11:12:09 PM	Success	Unread	312 bytes

Figure 34: Project Screenshot 34

The screenshot shows the Salesforce Flow Builder interface for a `Record-Triggered Flow`. The flow is configured to trigger when a `Claim` record is created or updated. The flow steps include `get_policy_records` (Get Records), `Claim_Filed_update` (Loop), and `Claim_Filed_state_update` (Update Records).

The `Configure Start` panel is open, showing the following configuration:

- Select Object:** `Claim`
- Configure Trigger:**
 - Trigger the Flow When:**
 - ☐ A record is created
 - ☐ A record is updated
 - ☒ A record is created or updated
 - ☐ A record is deleted
- Set Entry Conditions:** `None`

Figure 35: Project Screenshot 35

The screenshot displays the Salesforce Flow Builder interface for a Record-Triggered Flow named "Claim Policy Holder State Update - V1". The flow is configured as follows:

- Record-Triggered Flow:** Object: Claim, Trigger: A record is created or updated, Optimize for: Fast Field Updates.
- Get Records:** Label: get_policy_records, API Name: get_policy_records, Object: Policy.
- Loop:** Label: Claim_Filed_update, API Name: Claim_Filed_update, Condition Requirements: None—Get All Policy Records, Sort Order: Not Sorted, How Many Records to Store: Only the first record.
- Update Records:** Label: Claim_Filed_state_update, API Name: Claim_Filed_state_update.

The right sidebar shows the configuration for the "Get Records" action, including the label, API Name, Object, and various filters and sorting options.

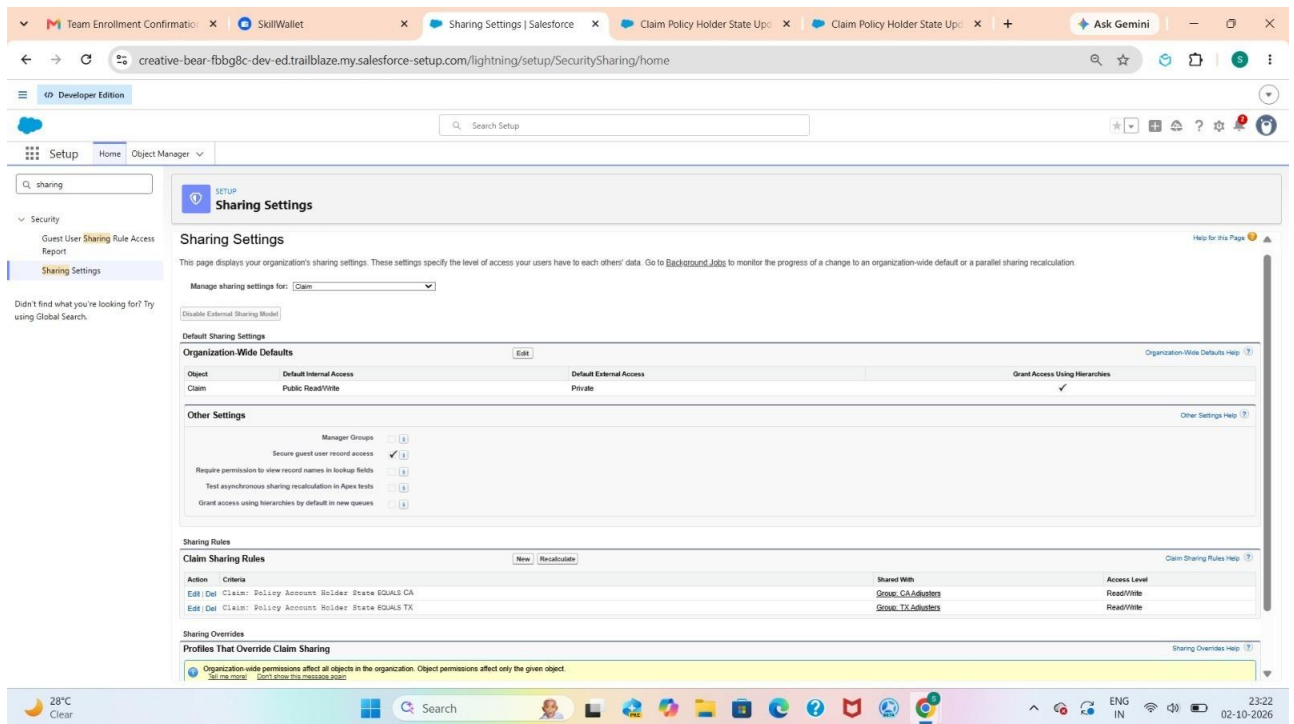
Figure 36: Project Screenshot 36

The screenshot displays the Salesforce Flow Builder interface for the same Record-Triggered Flow, but with the "Loop" action selected. The flow configuration is identical to Figure 35, but the right sidebar shows the configuration for the "Loop" action:

- Loop:** Label: Claim_Filed_update, API Name: Claim_Filed_update.
- Select Collection Variable:** Collection Variable: Policies from get_policy_records.
- Specify Direction for Iterating Over Collection:** Direction: First item to last item.

The right sidebar also includes a description of the Loop action and a note about using the current item in other elements in the loop.

Figure 37: Project Screenshot 37



This screenshot shows the Salesforce Sharing Settings page in the Developer Edition. The page is titled "Sharing Settings" and provides a comprehensive overview of an organization's sharing configuration. It includes sections for "Default Sharing Settings", "Other Settings", "Sharing Rules", and "Profiles That Override Claim Sharing".

Default Sharing Settings: This section allows users to manage sharing settings for a specific object (currently "Claim"). It includes options for "Default Internal Access" (Public Read/Write) and "Default External Access" (Private). A table below shows the "Grant Access Using Hierarchies" status for various objects.

Object	Default Internal Access	Default External Access	Grant Access Using Hierarchies
Claim	Public Read/Write	Private	✓

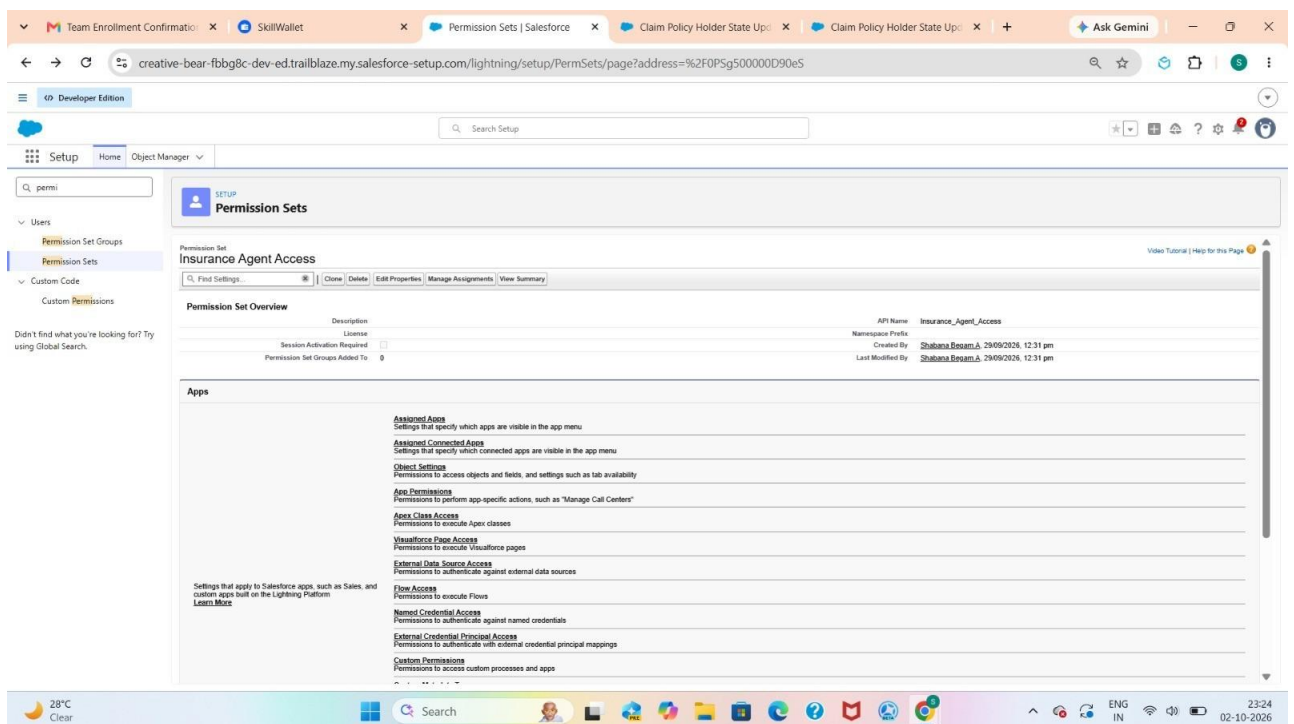
Other Settings: This section contains various checkboxes for sharing options, such as "Manager Groups", "Secure guest user record access", "Require permission to view record names in lookup fields", "Test asynchronous sharing recalculation in Apex tests", and "Grant access using hierarchies by default in new queries".

Sharing Rules: This section displays a list of sharing rules. A table below shows the details for a specific rule:

Action	Criteria	Shared With	Access Level
Edit	Del Claim: Policy Account Holder State EQUALS CA	Group: CA Adjusters	Read/Write
Edit	Del Claim: Policy Account Holder State EQUALS TX	Group: TX Adjusters	Read/Write

Profiles That Override Claim Sharing: This section provides information about profiles that override claim sharing, including a link to "Share Overrides Help".

Figure 38: Project Screenshot 38



This screenshot shows the Salesforce Permission Sets page in the Developer Edition. The page is titled "Permission Sets" and provides a comprehensive overview of an organization's permission set configuration. It includes sections for "Permission Set Overview", "Apps", and "Custom Permissions".

Permission Set Overview: This section displays a list of permission sets. A table below shows the details for a specific permission set:

API Name	Insurance Agent Access
Namespace Prefix	
Created By	Shobana Beem A. 29/09/2026, 12:31 pm
Last Modified By	Shobana Beem A. 29/09/2026, 12:31 pm

Apps: This section provides a detailed overview of the permission set's configuration, including sections for "Assigned Apps", "Assigned Connected Apps", "Object Settings", "App Permissions", "App Class Access", "Visualforce Page Access", "External Data Source Access", "Flow Access", "Named Credential Access", "External Credential Principal Access", and "Custom Permissions".

Figure 39: Project Screenshot 39

This screenshot shows the Salesforce Setup interface for a permission set named 'Claims Manager Access'. The left sidebar contains navigation links for Users, Permission Set Groups, Permission Sets (selected), Custom Code, and Custom Permissions. The main content area is titled 'Permission Set Overview' and includes a search bar, buttons for Clone, Delete, Edit Properties, Manage Assignments, and View Summary. Below this, a table displays metadata for the permission set, including its API Name, Namespace Prefix, Created By, and Last Modified By. The 'Apps' section lists various permission categories such as Assigned Apps, Assigned Connected Apps, Object Settings, App Permissions, Apex Class Access, Visualforce Page Access, External Data Source Access, Flow Access, Named Credential Access, External Credential Principal Access, and Custom Permissions. The bottom of the screen shows a Windows taskbar with the date and time as 23:24 on 02-10-2026.

Team Enrollment Confirmation x SkillWallet x Permission Sets | Salesforce x Claim Policy Holder State Up x Claim Policy Holder State Up x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/PermsSets/page?address=%2F0PSg500000DFJh

Developer Edition

Search Setup

Setup Home Object Manager

Q: permi

Users

Permission Set Groups

Permission Sets

Custom Code

Custom Permissions

Didn't find what you're looking for? Try using Global Search.

Permission Set

Claims Manager Access

Find Settings... Clone Delete Edit Properties Manage Assignments View Summary

Permission Set Overview

Description

License

Session Activation Required

Permission Set Groups Added To

API Name

Claims_Manager_Access

Namespace Prefix

Created By

Shabana Beem A. 02/10/2026, 9:33 pm

Last Modified By

Shabana Beem A. 02/10/2026, 9:43 pm

Apps

Assigned Apps

Settings that specify which apps are visible in the app menu

Assigned Connected Apps

Settings that specify which connected apps are visible in the app menu

Object Settings

Permissions to access objects and fields, and settings such as tab availability

App Permissions

Permissions to perform app-specific actions, such as "Manage Call Centers"

Apex Class Access

Permissions to execute Apex classes

Visualforce Page Access

Permissions to execute Visualforce pages

External Data Source Access

Permissions to authenticate against external data sources

Flow Access

Permissions to execute Flows

Named Credential Access

Permissions to authenticate against named credentials

External Credential Principal Access

Permissions to authenticate with external credential principal mappings

Custom Permissions

Permissions to access custom processes and apps

Settings that apply to Salesforce apps, such as Sales, and custom apps built on the Lightning Platform

Learn More

28°C Clear

Search

ENG IN

23:24 02-10-2026

Figure 40: Project Screenshot 40

This screenshot shows the Salesforce Setup interface for a permission set named 'Claims Adjuster Access'. The left sidebar contains navigation links for Users, Permission Set Groups, Permission Sets (selected), Custom Code, and Custom Permissions. The main content area is titled 'Permission Set Overview' and includes a search bar, buttons for Clone, Delete, Edit Properties, Manage Assignments, and View Summary. Below this, a table displays metadata for the permission set, including its API Name, Namespace Prefix, Created By, and Last Modified By. The 'Apps' section lists various permission categories such as Assigned Apps, Assigned Connected Apps, Object Settings, App Permissions, Apex Class Access, Visualforce Page Access, External Data Source Access, Flow Access, Named Credential Access, External Credential Principal Access, and Custom Permissions. The bottom of the screen shows a Windows taskbar with the date and time as 23:25 on 02-10-2026.

Team Enrollment Confirmation x SkillWallet x Permission Sets | Salesforce x Claim Policy Holder State Up x Claim Policy Holder State Up x + Ask Gemini

creative-bear-fbbg8c-dev-ed.trailblaze.my.salesforce-setup.com/lightning/setup/PermsSets/page?address=%2F0PSg500000DFJh

Developer Edition

Search Setup

Setup Home Object Manager

Q: permi

Users

Permission Set Groups

Permission Sets

Custom Code

Custom Permissions

Didn't find what you're looking for? Try using Global Search.

Permission Set

Claims Adjuster Access

Find Settings... Clone Delete Edit Properties Manage Assignments View Summary

Permission Set Overview

Description

License

Session Activation Required

Permission Set Groups Added To

API Name

Claims_Adjuster_Access

Namespace Prefix

Created By

Shabana Beem A. 02/10/2026, 9:44 pm

Last Modified By

Shabana Beem A. 02/10/2026, 9:45 pm

Apps

Assigned Apps

Settings that specify which apps are visible in the app menu

Assigned Connected Apps

Settings that specify which connected apps are visible in the app menu

Object Settings

Permissions to access objects and fields, and settings such as tab availability

App Permissions

Permissions to perform app-specific actions, such as "Manage Call Centers"

Apex Class Access

Permissions to execute Apex classes

Visualforce Page Access

Permissions to execute Visualforce pages

External Data Source Access

Permissions to authenticate against external data sources

Flow Access

Permissions to execute Flows

Named Credential Access

Permissions to authenticate against named credentials

External Credential Principal Access

Permissions to authenticate with external credential principal mappings

Custom Permissions

Permissions to access custom processes and apps

Settings that apply to Salesforce apps, such as Sales, and custom apps built on the Lightning Platform

Learn More

28°C Clear

Search

ENG IN

23:25 02-10-2026